

Practical 2 — Testing IID and Gaussian Characters

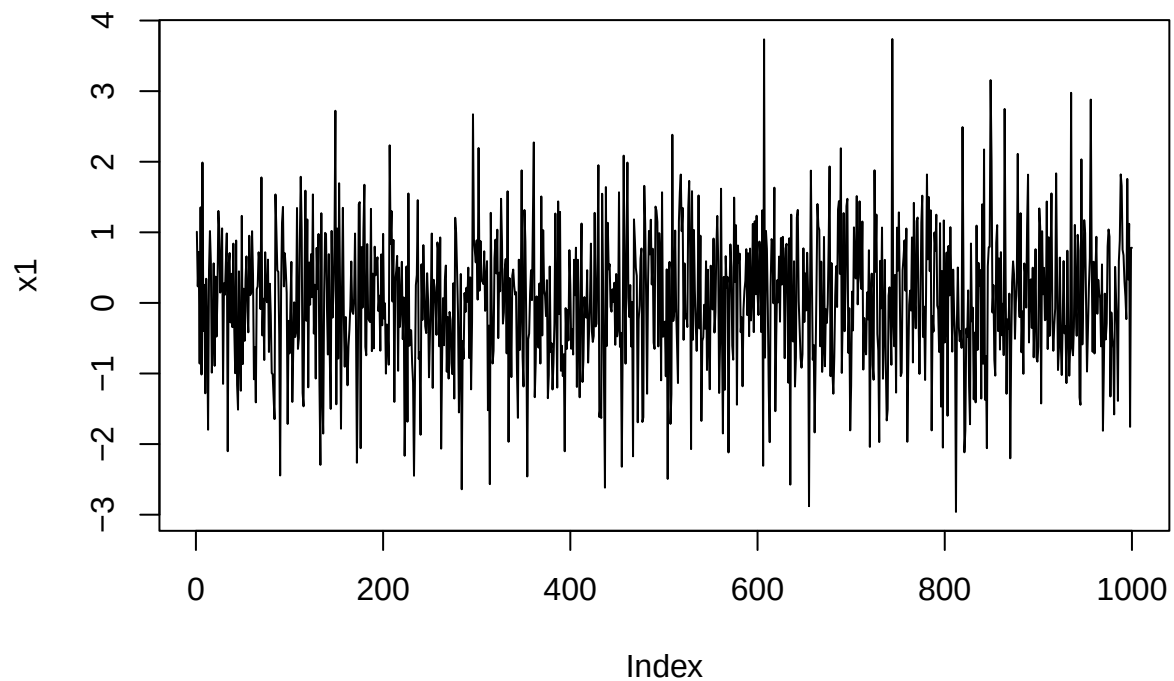
Practical 2A

1. Generate samples of length n of different IID noises using the following instructions

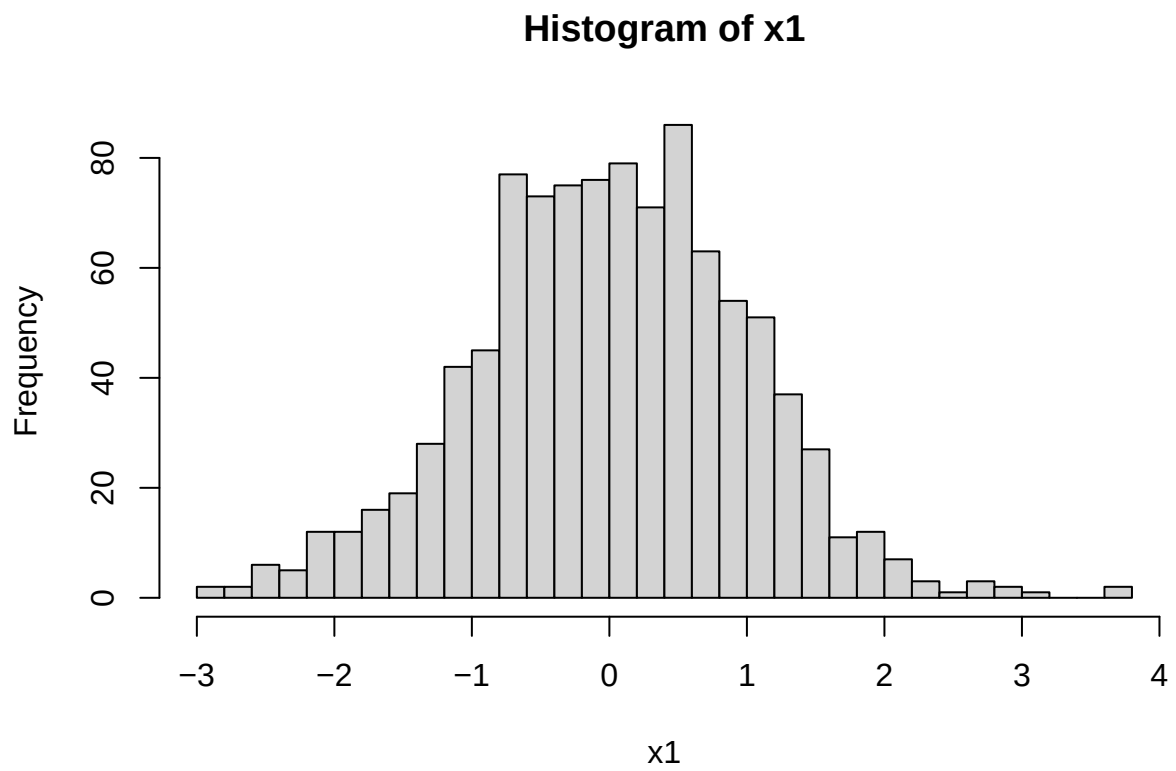
```
n <- 1000

x1 <- rnorm(n, mean = 0, sd = 1)
x2 <- runif(n, min = 0, max = 1)
x3 <- rexp(n, rate = 2)
x4 <- rchisq(n, df = 3)
x5 <- rpois(n, lambda = 4)
x6 <- rt(n, df = 5)

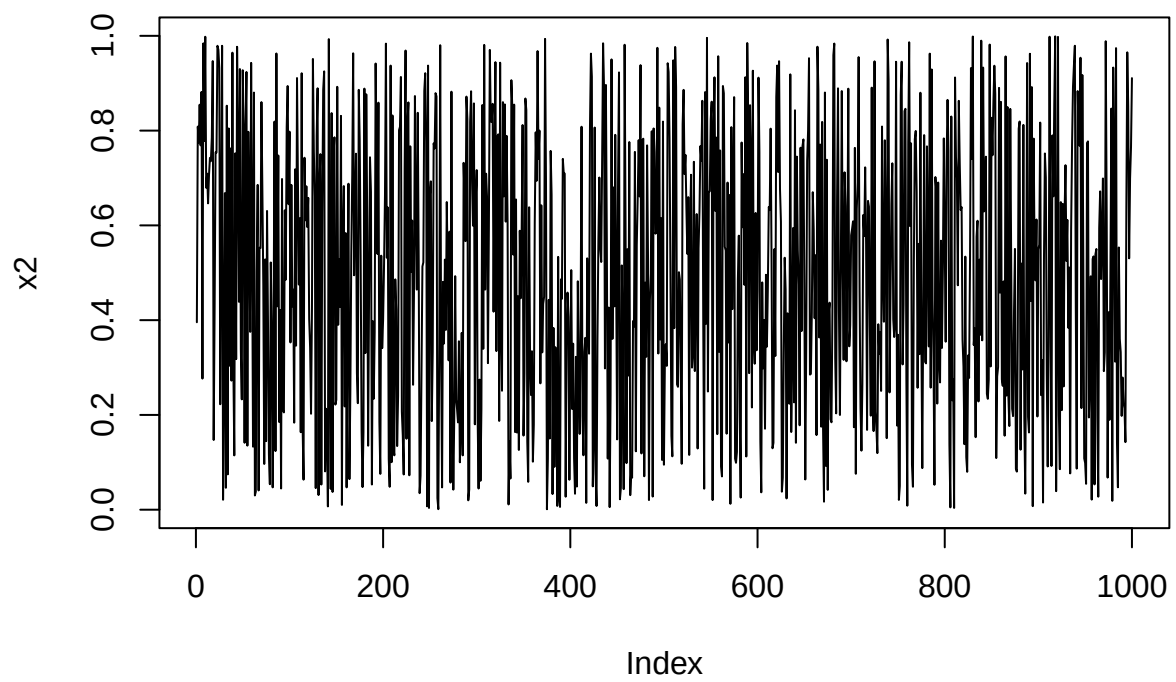
plot(x1, type="l")
```



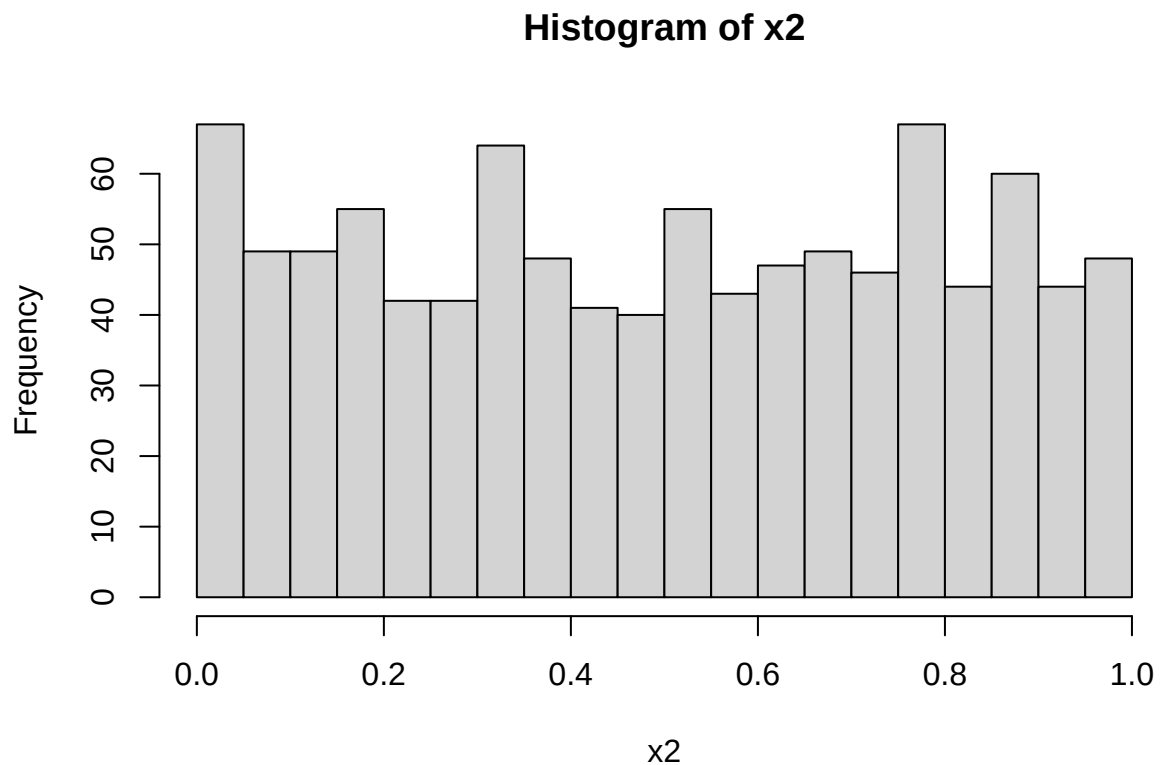
```
hist(x1, 30)
```



```
plot(x2, type="l")
```



```
hist(x2, 30)
```



Observe what happens when you move the parameters. Try different distributions and parameters. What do you observe?

Normal distribution: change mean and sd

```
n <- 1000

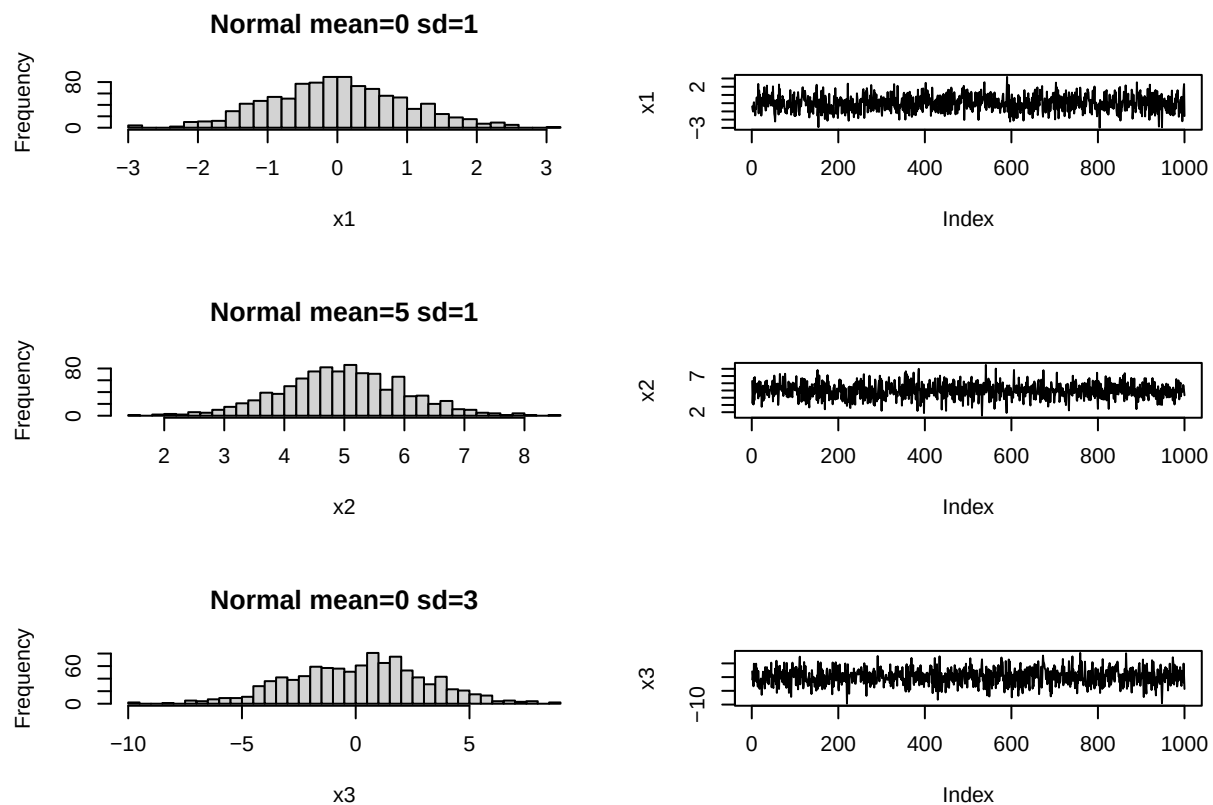
x1 <- rnorm(n, mean = 0, sd = 1)
x2 <- rnorm(n, mean = 5, sd = 1)
x3 <- rnorm(n, mean = 0, sd = 3)

par(mfrow=c(3,2))

hist(x1, 30, main="Normal mean=0 sd=1")
plot(x1, type="l")

hist(x2, 30, main="Normal mean=5 sd=1")
plot(x2, type="l")

hist(x3, 30, main="Normal mean=0 sd=3")
plot(x3, type="l")
```



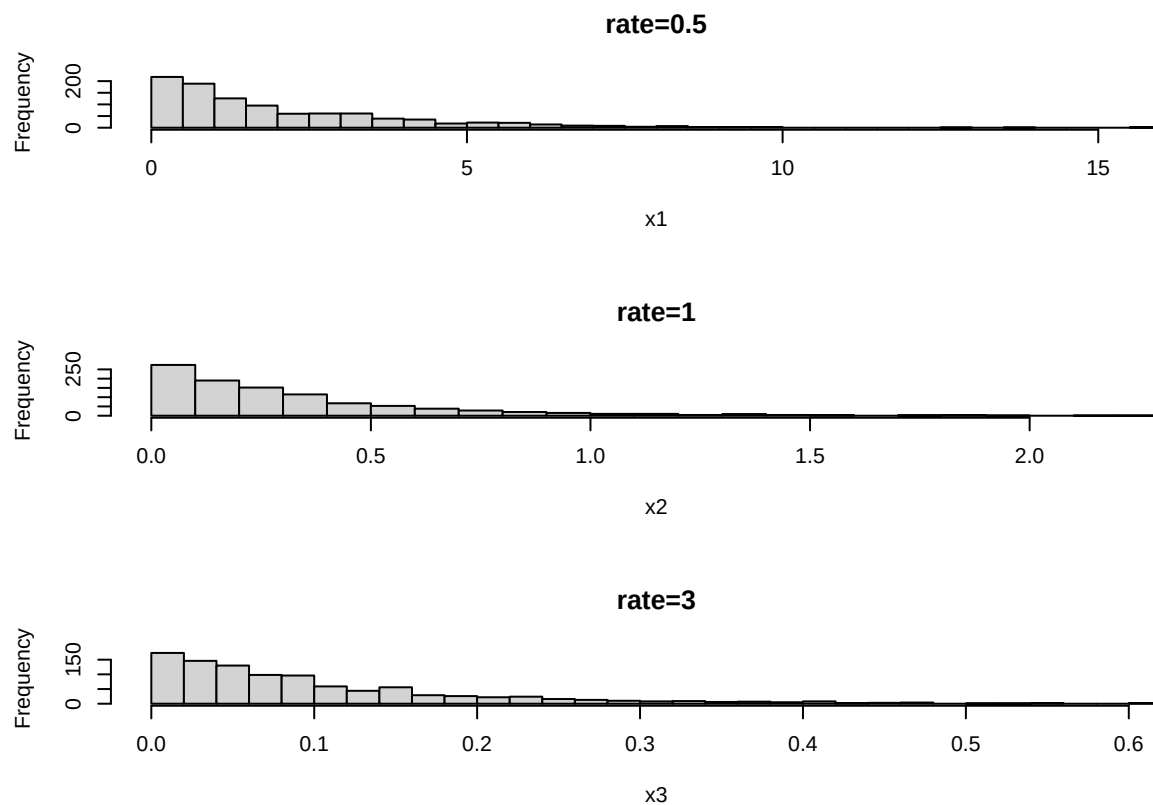
We see that: - Increasing the mean shift the distribution - Increasing sd spreads the data

Exponential distribution: change rate

```
x1 <- rexp(n, rate=0.5)
x2 <- rexp(n, rate=3)
x3 <- rexp(n, rate=10)

par(mfrow=c(3,1))

hist(x1, 30, main="rate=0.5")
hist(x2, 30, main="rate=1")
hist(x3, 30, main="rate=3")
```



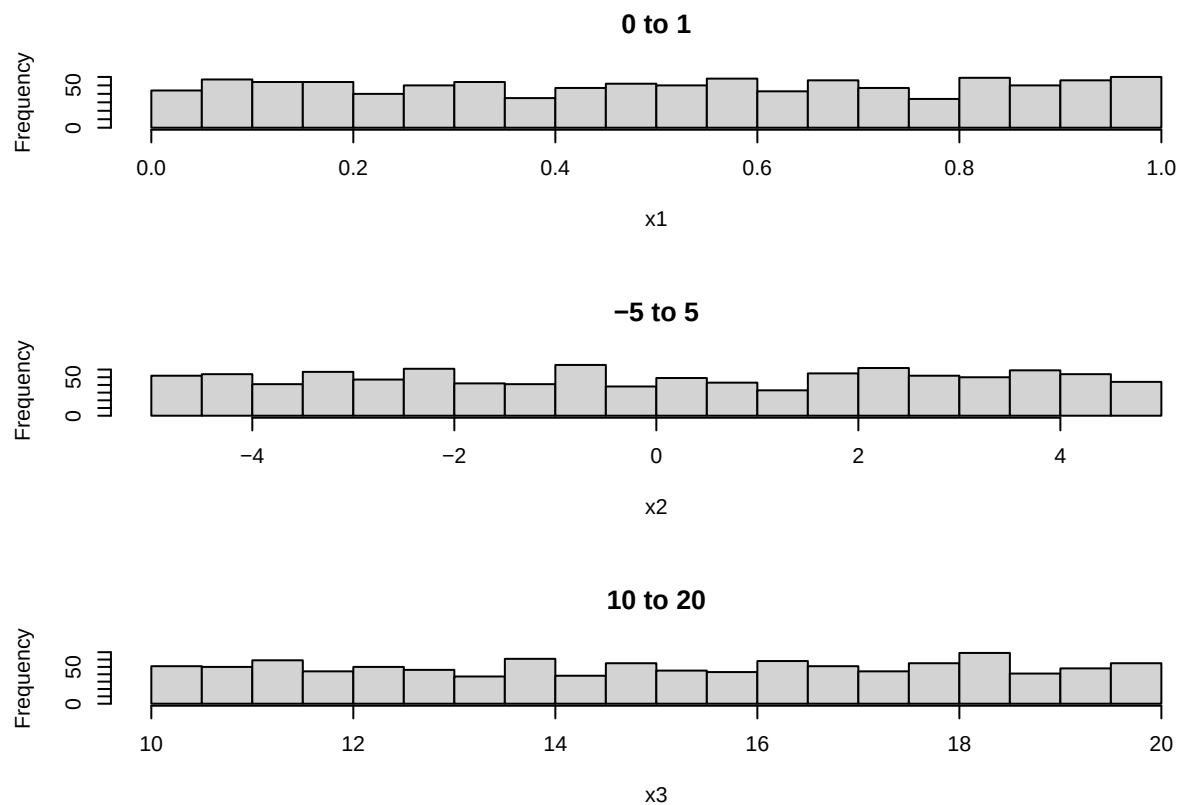
We see that: - Higher rate implies that values are more concentrated around 0 - Lower rate implies that values are more spread out, longer

Uniform distribution: change rate

```
x1 <- runif(n, min=0, max=1)
x2 <- runif(n, min=-5, max=5)
x3 <- runif(n, min=10, max=20)

par(mfrow=c(3,1))

hist(x1, 30, main="0 to 1")
hist(x2, 30, main="-5 to 5")
hist(x3, 30, main="10 to 20")
```



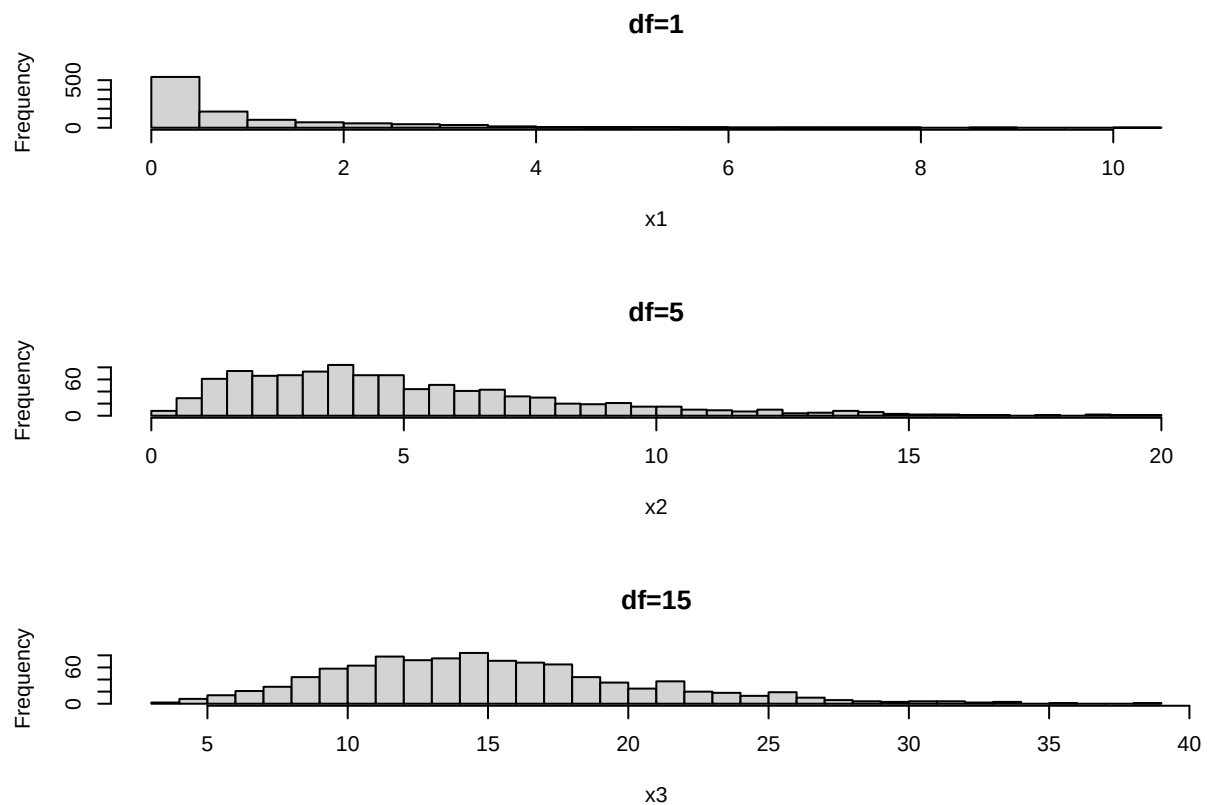
We see that the distribution shape is the same, but the values are shifted and more spread out when we increase the range.

Chi-square: change the degrees of freedom

```
x1 <- rchisq(n, df=1)
x2 <- rchisq(n, df=5)
x3 <- rchisq(n, df=15)

par(mfrow=c(3,1))

hist(x1, 30, main="df=1")
hist(x2, 30, main="df=5")
hist(x3, 30, main="df=15")
```



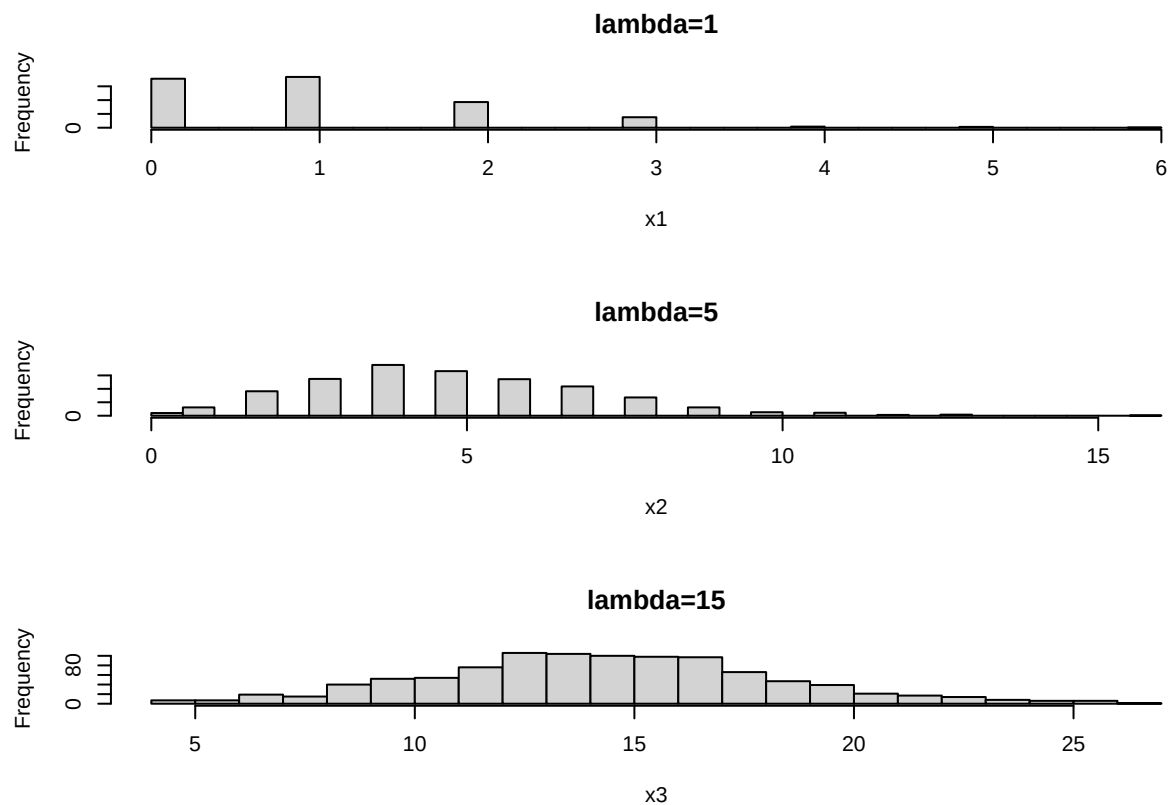
We see that: - Small df implies that values are more concentrated around 0, with a long tail - Large df implies that values are more spread out, with a shorter tail

Poisson distribution: change lambda

```
x1 <- rpois(n, lambda=1)
x2 <- rpois(n, lambda=5)
x3 <- rpois(n, lambda=15)

par(mfrow=c(3,1))

hist(x1, 30, main="lambda=1")
hist(x2, 30, main="lambda=5")
hist(x3, 30, main="lambda=15")
```

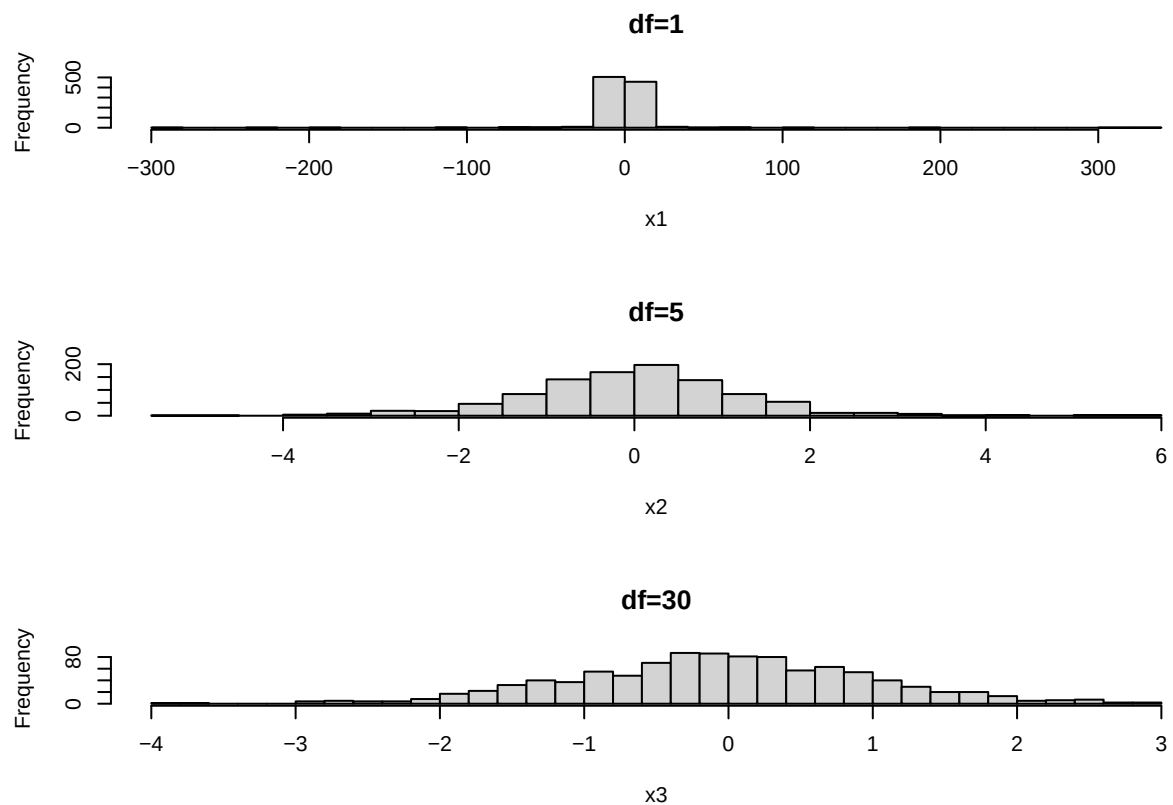
We see that: - Small λ implies that values are more concentrated around 0, with a long tail - Large λ implies that values are more spread out, with a shorter tail

t distribution: change the degrees of freedom

```
x1 <- rt(n, df=1)
x2 <- rt(n, df=5)
x3 <- rt(n, df=30)

par(mfrow=c(3,1))

hist(x1, 30, main="df=1")
hist(x2, 30, main="df=5")
hist(x3, 30, main="df=30")
```

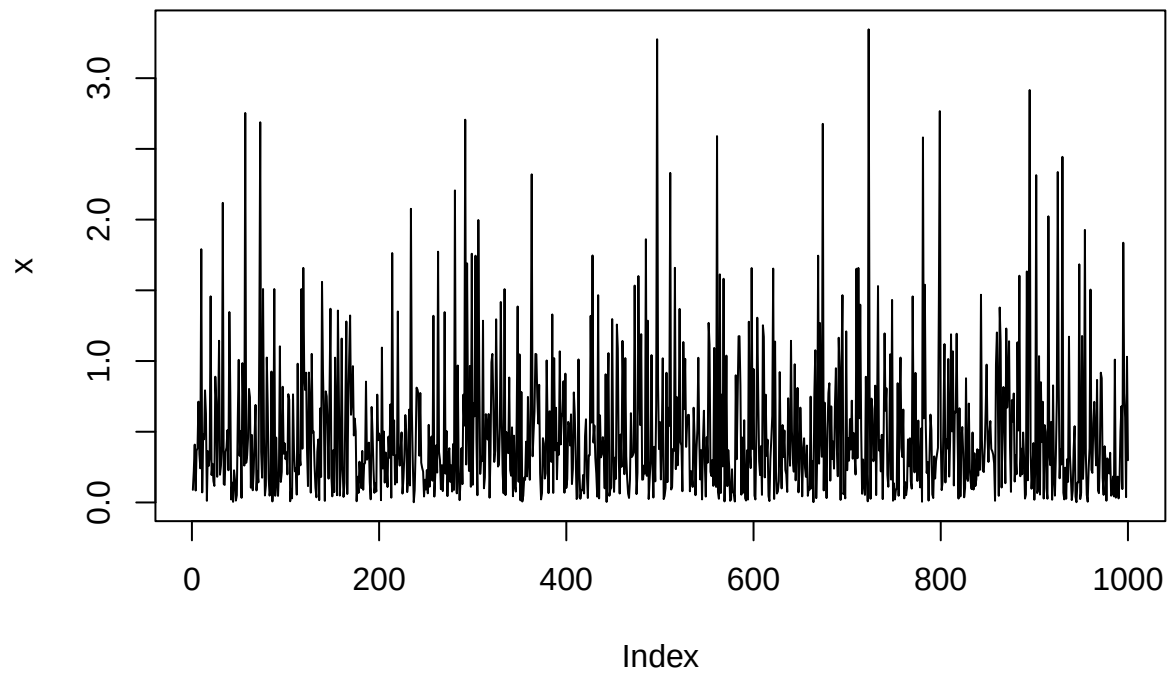


We see that: - Small df implies that values are more concentrated around 0, with a long tail - Large df implies that values are more spread out, with a shorter tail, and when df is large the distribution looks more like a normal distribution

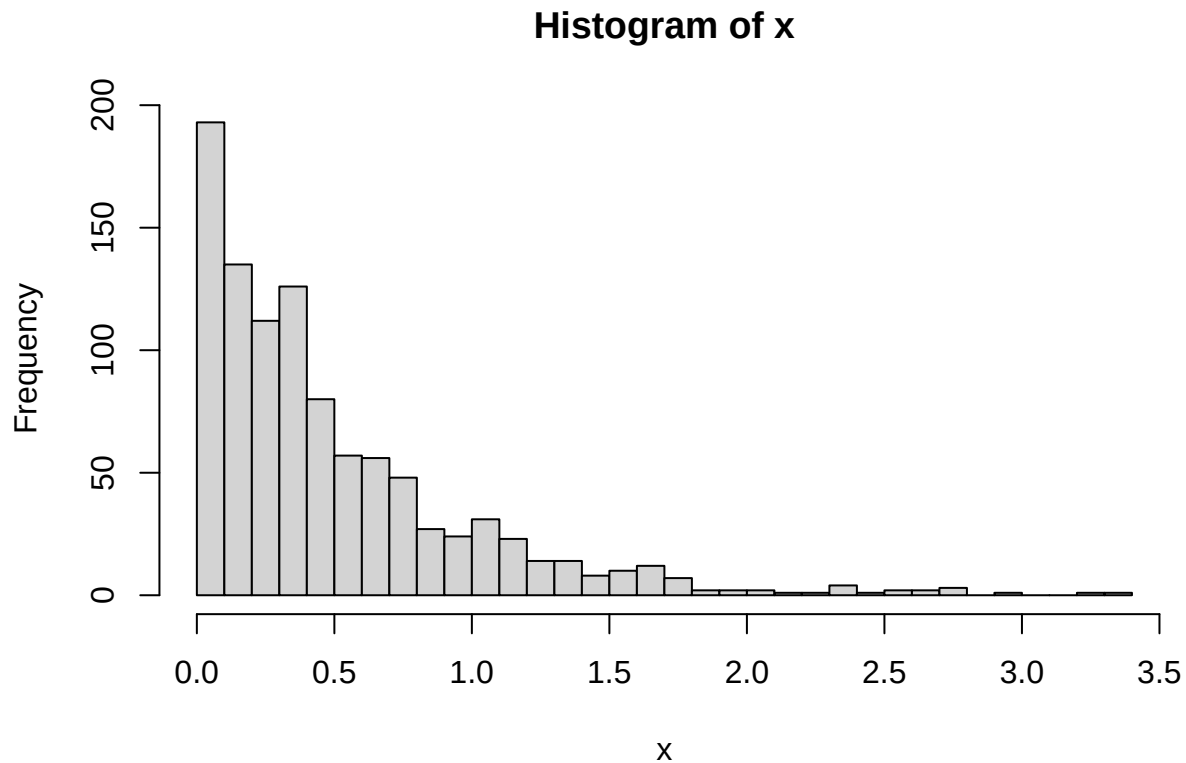
Conclusion: moving the parameters change - location -> mean - spread -> variance/sd - skewness -> rate/df - tail -> rate/df

Observe what happens when you move the parameters. For different distributions, do for example

```
n <- 1000
k <- 30
x <- rexp(n, rate=2)
plot(x, type="l")
```



```
hist(x, k)
```



Here k denotes the number of classes for the histogram. Test with other distributions writing distributions in the help utility.

To explore all distributions in R:

```
help("Distributions")
```

or

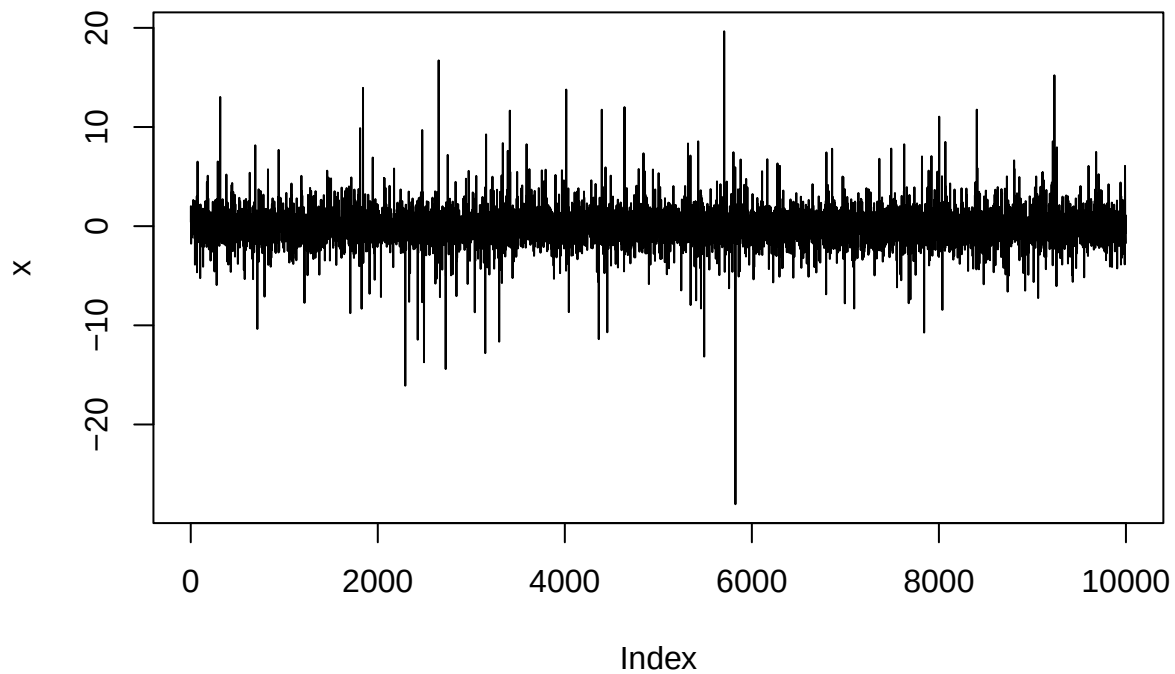
```
?rexp  
?rnorm
```

2. Verify the IID character of any sample of the previous exercise. Use Ljung-Box test with the instruction

```
h <- 1  
Box.test(x, lag=h, type="Ljung")  
  
##  
## Box-Ljung test  
##  
## data: x  
## X-squared = 3.1742, df = 1, p-value = 0.07481
```

For example:

```
n <- 10000
k <- 3
x <- rt(n, df=k)
plot(x, type="l")
```



```
Box.test(x, lag=1, type="Ljung")
```

```
##
## Box-Ljung test
##
## data: x
## X-squared = 1.4055, df = 1, p-value = 0.2358
```

Interpretation: We see a large p-value, which means that we cannot reject the null hypothesis of independence, so we can conclude that the data are independent. The sample behaves like IID noise, as expected.

We can try different lags, from $h=1$, to $\log(n)$:

```
for(h in 1:log(n)){
  print(Box.test(x, lag=h, type="Ljung"))
}
```

```
##
```

```

## Box-Ljung test
##
## data:  x
## X-squared = 1.4055, df = 1, p-value = 0.2358
##
##
## Box-Ljung test
##
## data:  x
## X-squared = 2.0421, df = 2, p-value = 0.3602
##
##
## Box-Ljung test
##
## data:  x
## X-squared = 2.0665, df = 3, p-value = 0.5587
##
##
## Box-Ljung test
##
## data:  x
## X-squared = 5.5784, df = 4, p-value = 0.2329
##
##
## Box-Ljung test
##
## data:  x
## X-squared = 5.7935, df = 5, p-value = 0.3268
##
##
## Box-Ljung test
##
## data:  x
## X-squared = 6.7064, df = 6, p-value = 0.3489
##
##
## Box-Ljung test
##
## data:  x
## X-squared = 7.059, df = 7, p-value = 0.4228
##
##
## Box-Ljung test
##
## data:  x
## X-squared = 7.1699, df = 8, p-value = 0.5184
##
##
## Box-Ljung test
##
## data:  x
## X-squared = 10.669, df = 9, p-value = 0.2991

```

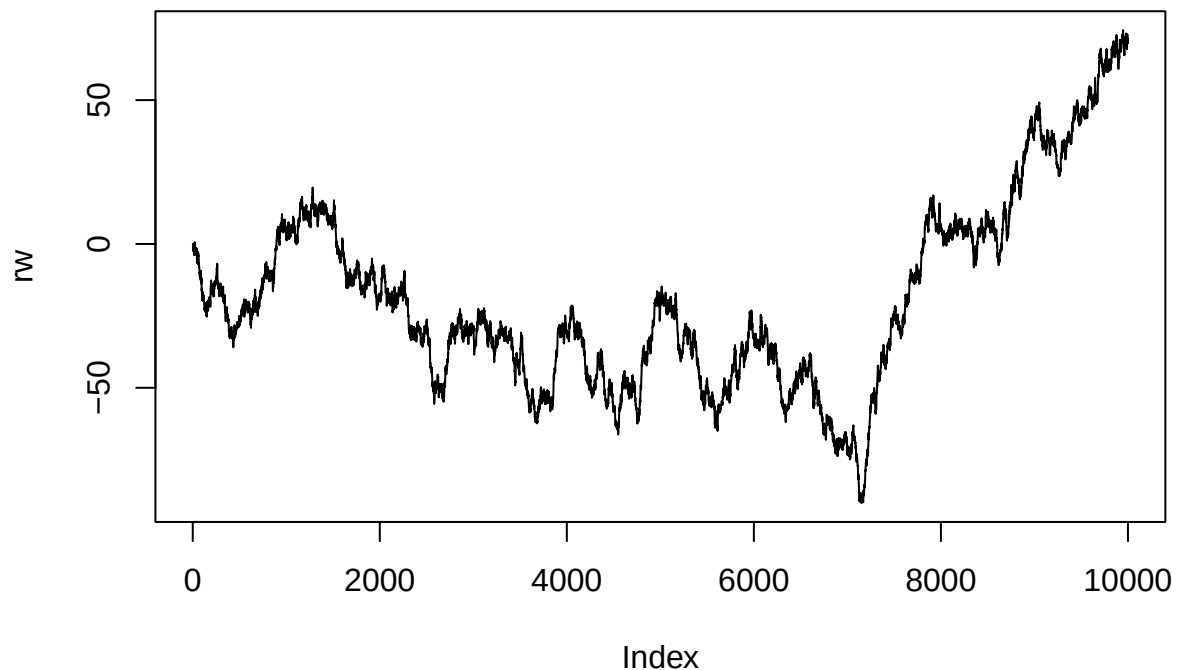
We see that for all lags, we cannot reject the null hypothesis of independence, so we can conclude that the

data are independent. p-values are all large, confirming that the data are independent.

Comparison with random walk

A random walk is not IID because each value depends on the previous one. We can verify this by applying the Ljung-Box test to a random walk:

```
n <- 10000
rw <- cumsum(rnorm(n))
plot(rw, type="l")
```



```
Box.test(rw, lag=log(n), type="Ljung")
```

```
##
## Box-Ljung test
##
## data:  rw
## X-squared = 89298, df = 9.2103, p-value < 2.2e-16
```

We see a small p-value, which means that we reject the null hypothesis of independence, so we can conclude that the data are not independent. The random walk does not behave like IID noise, as expected.

We can try different lags for the random walk as well:

```
rw <- cumsum(rnorm(n))
for(h in 1:log(n)){
  print(Box.test(rw, lag=h, type="Ljung"))
}
```

```
##
## Box-Ljung test
##
## data: rw
## X-squared = 9981.2, df = 1, p-value < 2.2e-16
##
##
## Box-Ljung test
##
## data: rw
## X-squared = 19941, df = 2, p-value < 2.2e-16
##
##
## Box-Ljung test
##
## data: rw
## X-squared = 29881, df = 3, p-value < 2.2e-16
##
##
## Box-Ljung test
##
## data: rw
## X-squared = 39800, df = 4, p-value < 2.2e-16
##
##
## Box-Ljung test
##
## data: rw
## X-squared = 49698, df = 5, p-value < 2.2e-16
##
##
## Box-Ljung test
##
## data: rw
## X-squared = 59576, df = 6, p-value < 2.2e-16
##
##
## Box-Ljung test
##
## data: rw
## X-squared = 69434, df = 7, p-value < 2.2e-16
##
##
## Box-Ljung test
##
## data: rw
## X-squared = 79271, df = 8, p-value < 2.2e-16
##
```



```
##
## Box-Ljung test
##
## data:  rw
## X-squared = 89088, df = 9, p-value < 2.2e-16
```

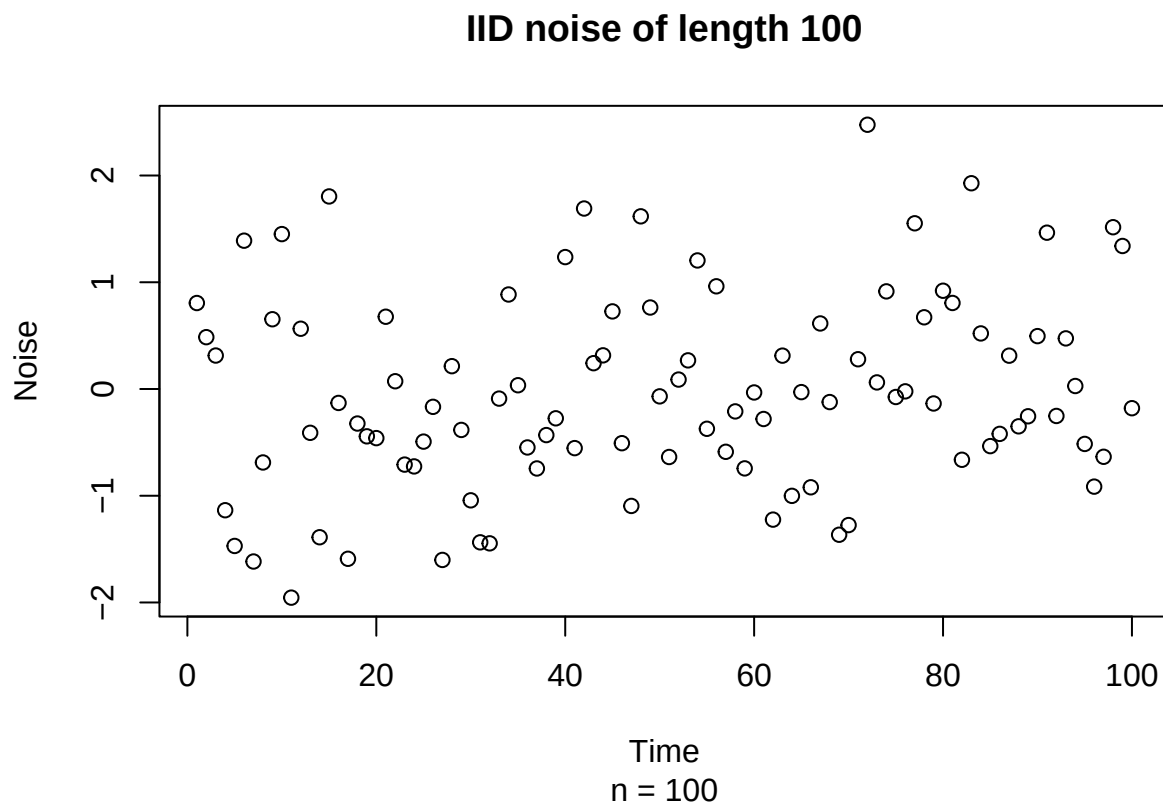
We see that for all lags, we reject the null hypothesis of independence, so we can conclude that the data are not independent. p-values are all small, confirming that the data are not independent.

Practical 2B

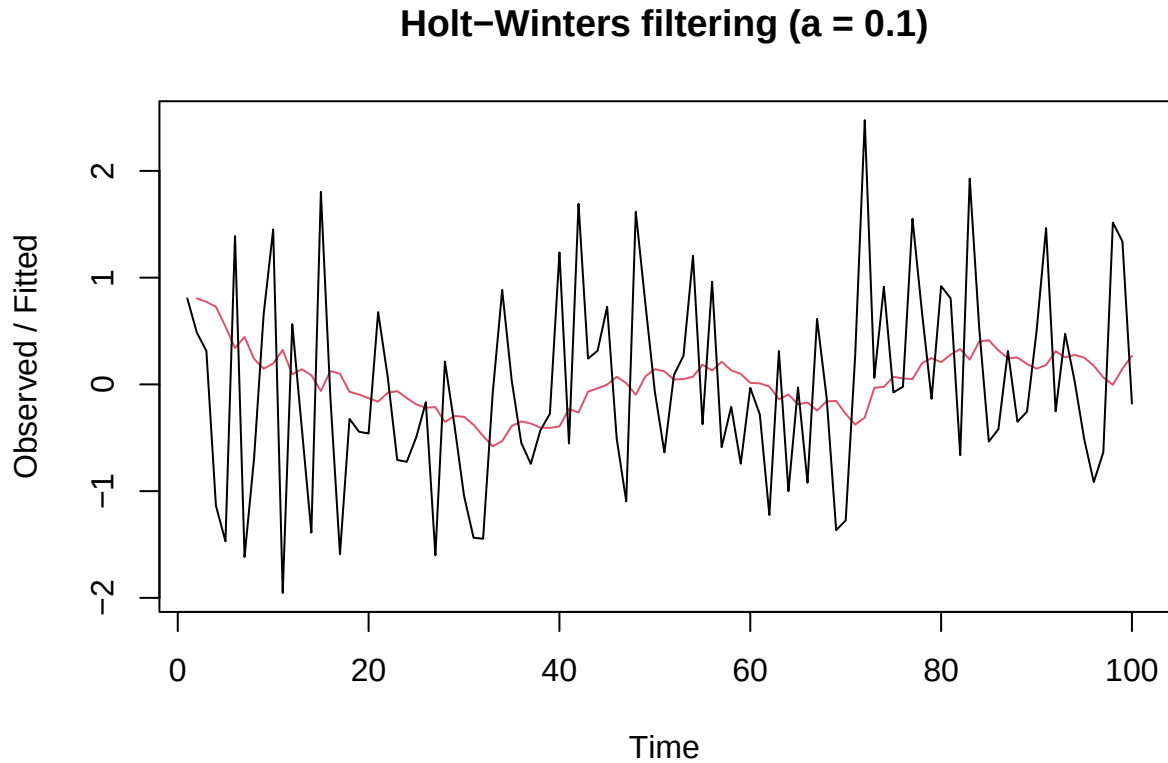
1. Generate an IID noise of length n and find its exponential smoothing series

The following piece of code generates the Holt Winters smoothing for stationary series with no trend and no seasonality.

```
n <- 100
x <- rnorm(n)
plot(x,
     main = "IID noise of length 100",
     sub = "n = 100",
     xlab = "Time",
     ylab = "Noise")
```



```
a <- 0.1
y <- HoltWinters(x,alpha=a,beta=F,gamma=F)
plot(y, main = "Holt-Winters filtering (a = 0.1)")
```



The generated series is an IID noise, so it does not have any systematic trend or seasonal component.

When generating the HoltWinters filtering, we set `beta=FALSE` and `gamma=FALSE`, which implies that the Holt-Winters method simply applies an exponential smoothing, which estimates a smoothed level of the series. In fact, the filtered signal, which is **red**, oscillates around the mean of the noise and removes part of the high-frequency variability. Next we will see how this behavior of removing the high-frequency components changes as we vary “a”.

1.1 Try for different values of: “n” and “a”

First we create a function to plot 6 subplots in one view, so that we can easily visualize how the filter changes when “a” varies:

```
hw_noise_plots <- function(n){
  set.seed(123) # we fix the seed so that we have the same noise each time we run the code
  # generate IID noise
  x <- rnorm(n)
  # alpha values
```

```

alphas <- c(0.01, 0.1, 0.2, 0.5, 0.8, 0.9)

# layout
par(
mfrow = c(2,3),
oma = c(0,0,3,0), # outer margin for global title
mar = c(4,4,2,1) # bottom, left, top, right margins
)

# loop over alphas
for(a in alphas){

  y <- HoltWinters(x, alpha = a, beta = FALSE, gamma = FALSE)

  plot(y, main = paste("a =", a))
}

# global title
mtext(paste("Holt-Winters filtering with n =", n),
      outer = TRUE, cex = 1.3)
}

```

For n high (e.g. $n > 200$) it's hard to see the filter signal on top of the original time series because the time series is very dense, so its black color prevails over the red. For this reason, we create another function to plot the original series in **grey**, and add on top the filtered signal in **red**:

```

hw_noise_plots_n_high <- function(n,x){

  set.seed(123)
  x <- rnorm(n)

  alphas <- c(0.01, 0.1, 0.2, 0.5, 0.8, 0.9)

  par(
mfrow = c(2,3),
oma = c(0,0,3,0), # outer margin for global title
mar = c(4,4,2,1) # bottom, left, top, right margins
)

  for(a in alphas){

    y <- HoltWinters(x, alpha=a, beta=FALSE, gamma=FALSE)
    ylim <- range(x) # to fix same y-scale in the plot

    # plot noise
    plot(x,
         ylim = ylim,
         type="l",
         col="grey",
         main=paste("alpha =", a),
         xlab="Time",
         ylab="Value")

```

```

# add smoothed signal
lines(y$fitted[,1], col="red", lwd=2)
}

mtext(paste("Holt-Winters filtering with n =", n),
      outer=TRUE, cex=1.3)
}

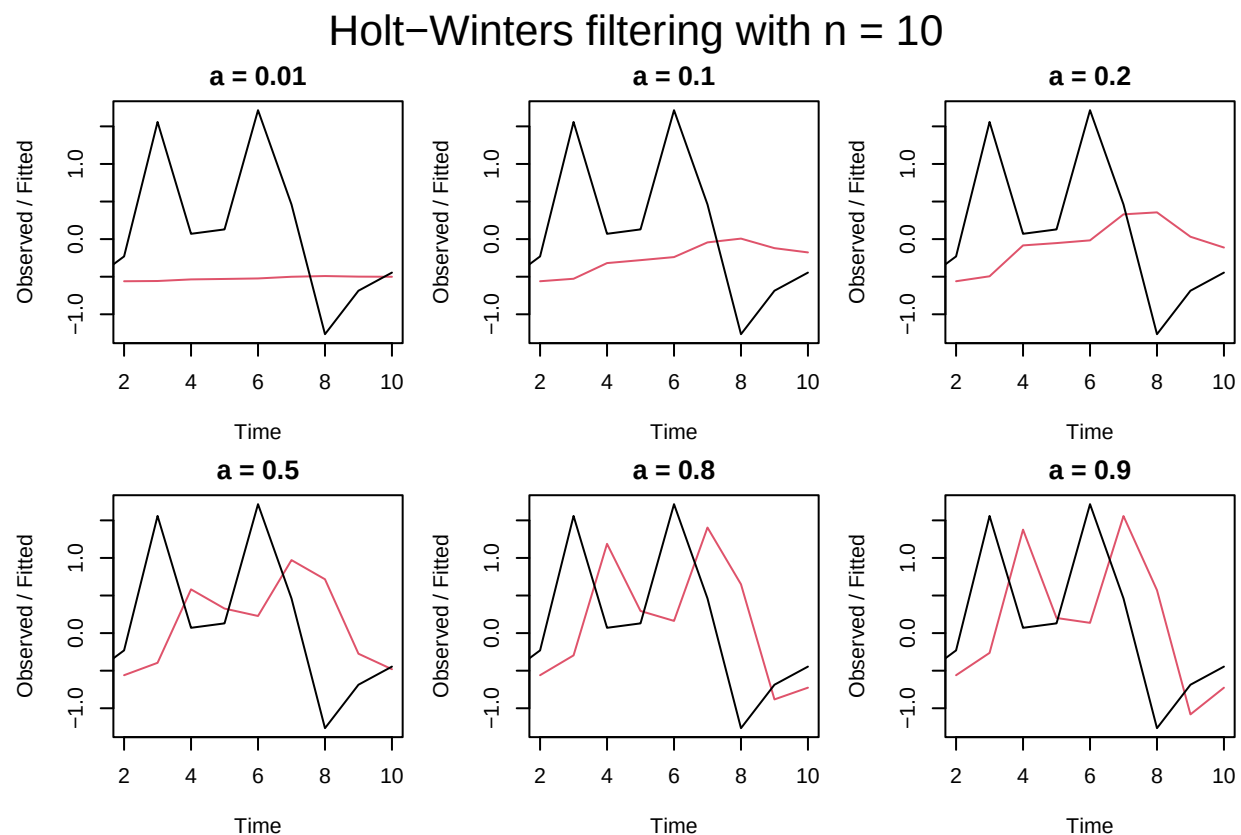
```

Now we will try to vary “a” between 0 and 1, for different amount of data points n (from 10 to 1000). The aim is to compare how the filtering behavior changes with “a” small or large:

```

# n = 10, a varies between 0 and 1
hw_noise_plots(10)

```

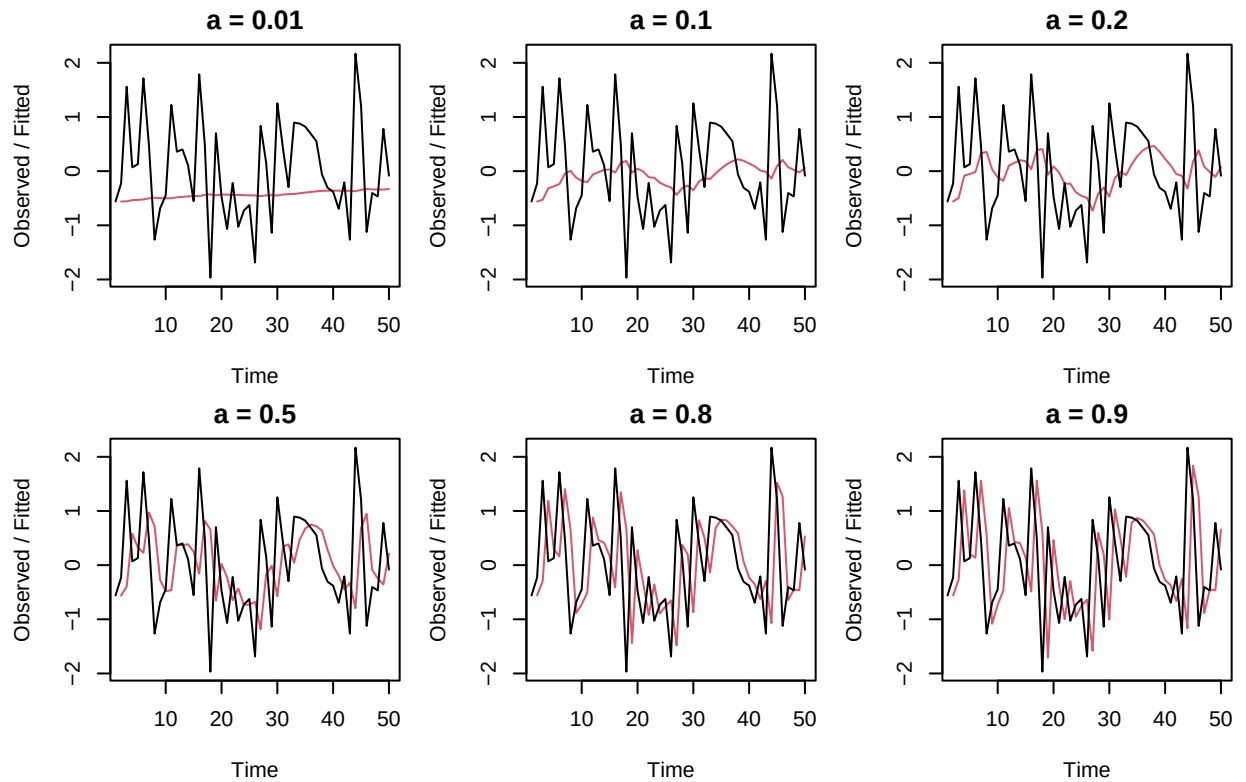


```

# n = 50, a varies between 0 and 1
hw_noise_plots(50)

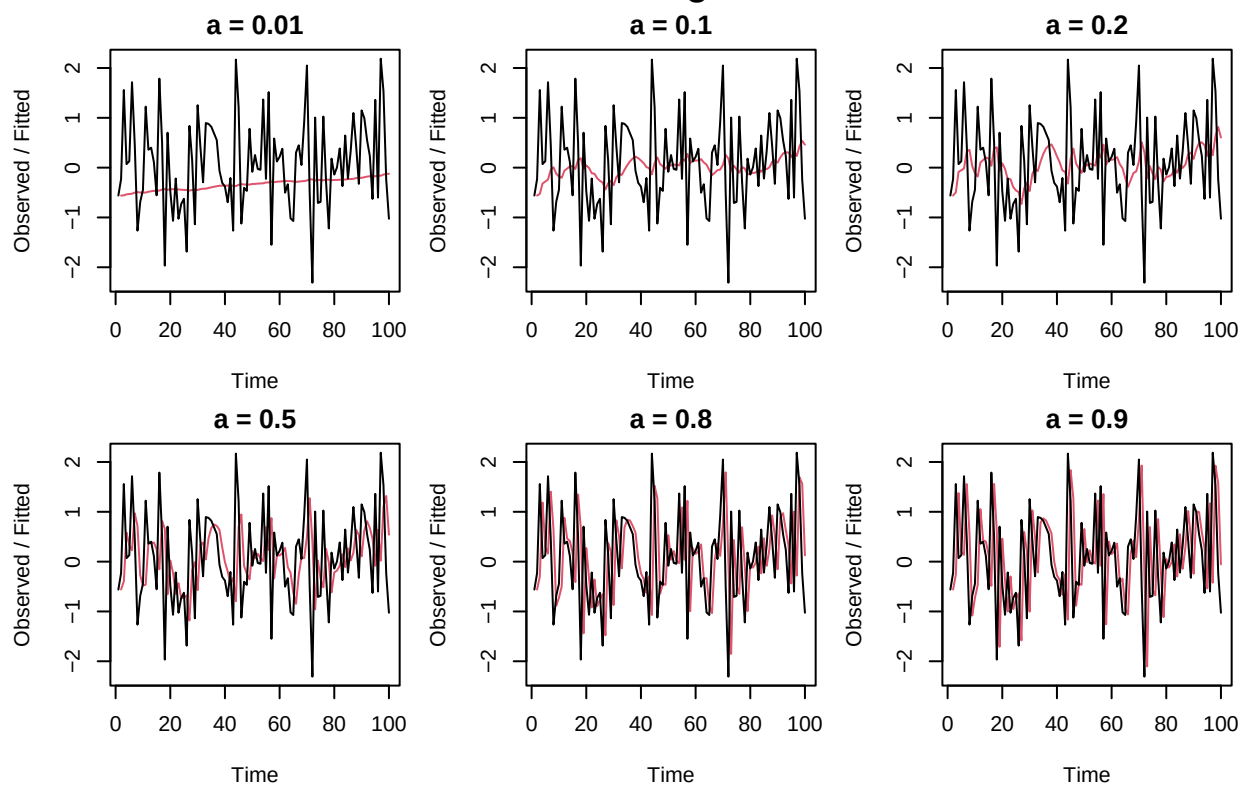
```

Holt-Winters filtering with $n = 50$



```
# n = 100, a varies between 0 and 1  
hw_noise_plots(100)
```

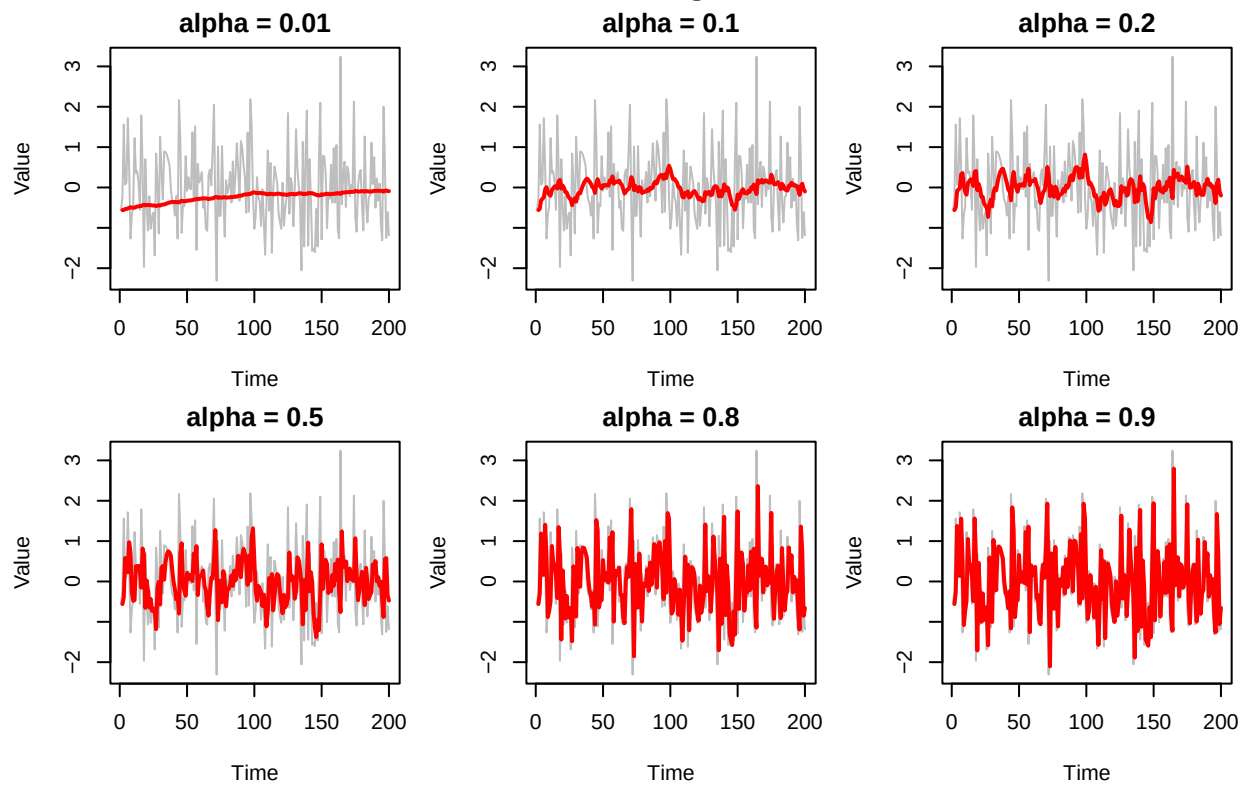
Holt-Winters filtering with $n = 100$



$n = 200$, α varies between 0 and 1

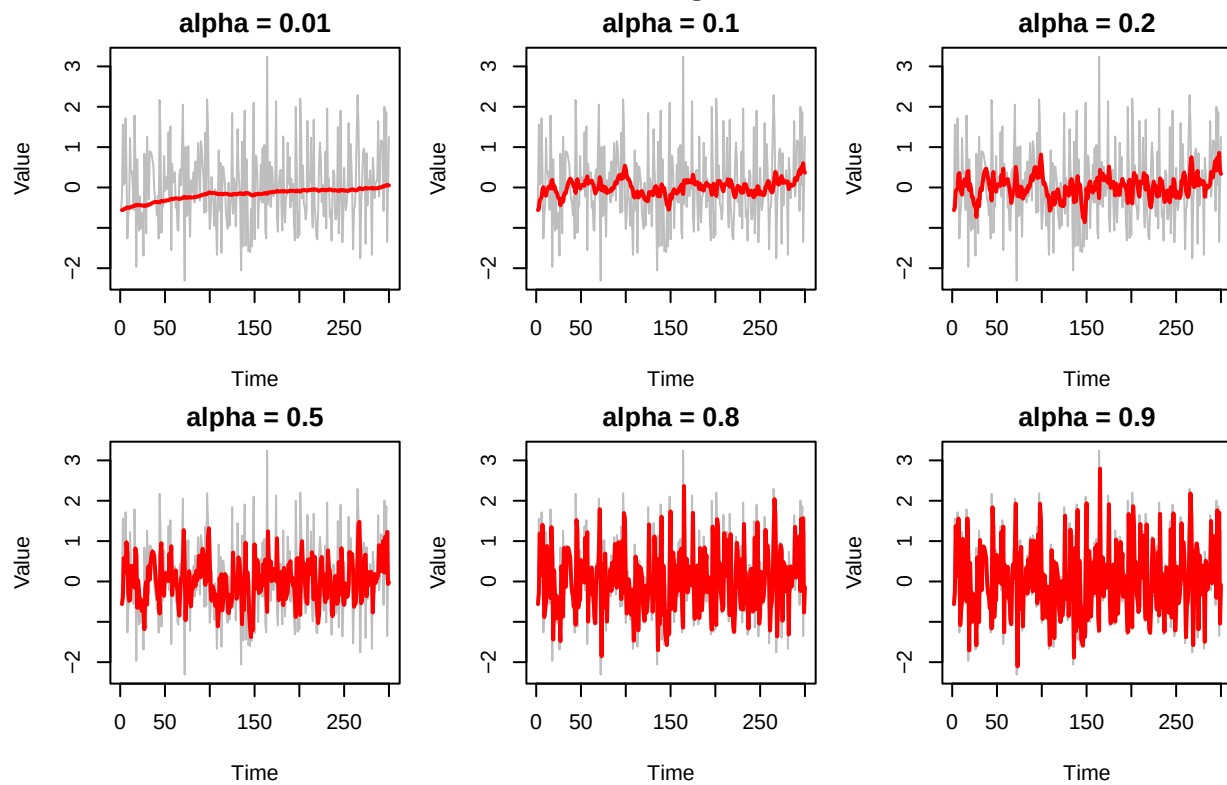
`hw_noise_plots_n_high(200)` *# I use the function to plot the original series in grey so that I can clear*

Holt-Winters filtering with $n = 200$



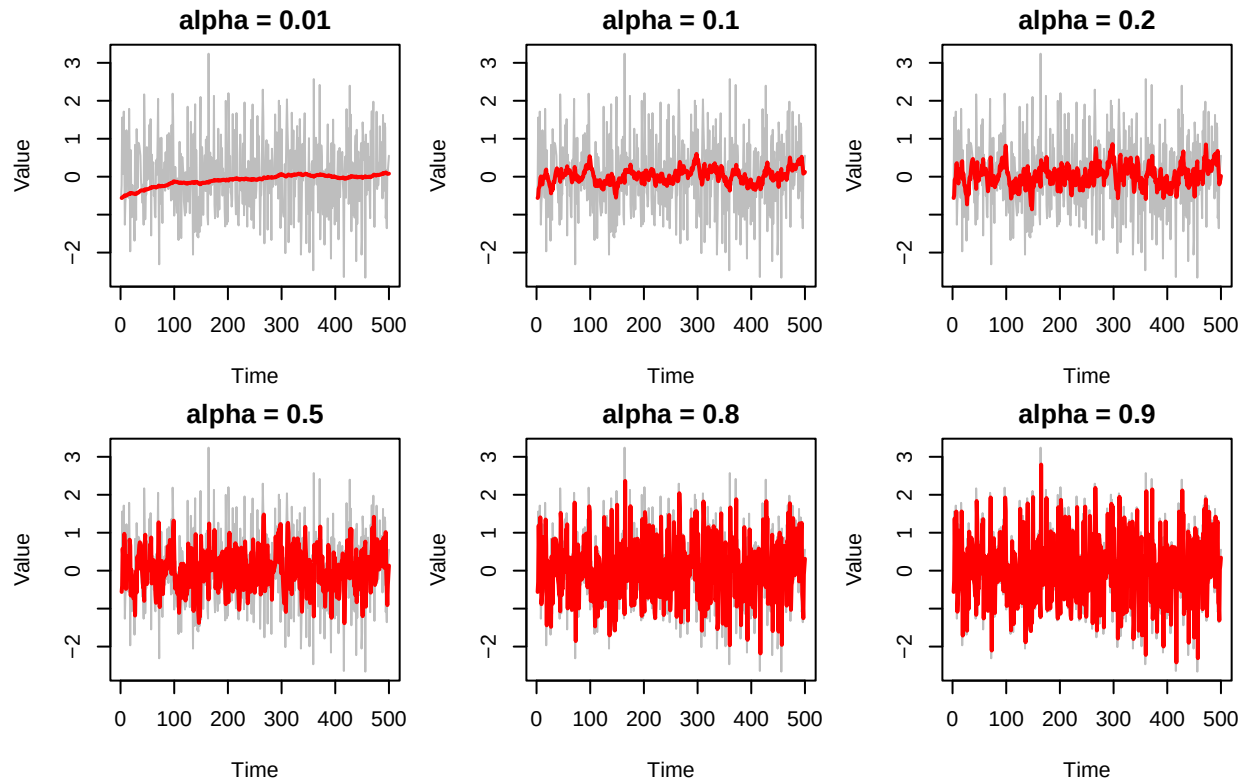
```
#  $n = 300$ ,  $\alpha$  varies between 0 and 1  
hw_noise_plots_n_high(300)
```

Holt-Winters filtering with $n = 300$



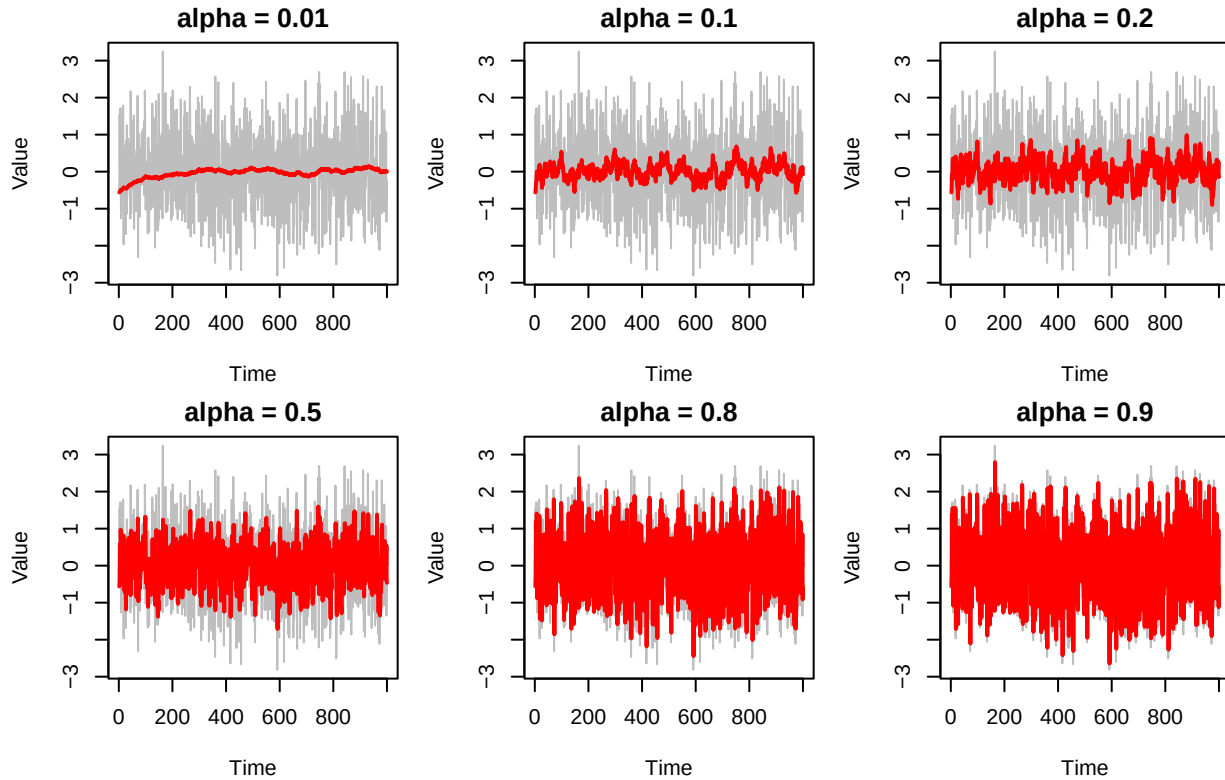
```
#  $n = 500$ ,  $\alpha$  varies between 0 and 1  
hw_noise_plots_n_high(500)
```


Holt-Winters filtering with $n = 500$



```
#  $n = 1000$ ,  $a$  varies between 0 and 1  
hw_noise_plots_n_high(1000)
```

Holt-Winters filtering with $n = 1000$



Observations:

Changing “a”: As it is confirmed by the plots above, the parameter “a” controls how smooth is the estimation of the series:

- Small values of “a” produce a very smooth estimate
- Larger values of “a” produce a very noisy estimate, that tends to match the original series

This is coherent with the fact that a small “a” gives more weight to past observations, while a large “a” gives more weight to recent observations and so the filtered series reacts quickly to changes in noise - and therefore for “a” very large the filtered series almost matches the original series.

Changing n: this doesn’t change the statistical properties of the IID noise itself, but it’s more a matter of visualization and observation:

- for n small, we can see the filtered series much more clearly because the plotted signal is not dense; so it’s easier to visualize how far is the filtered signal from the original series. But on the other side, we can observe the behavior of the smoothing filter for only few data points (e.g. 50);
- for n large, the plotted signal is visualized as much more dense so it’s not easy to see all the peaks and variations of the noise in detail, especially when “a” increases as well. So it may be harder to visualize the effect of the smoothing filter, but on the other hand it allows to observe the behavior of the filter over longer periods of time (i.e. for more data points, e.g. 500, 1000).

1.2 Simulate other IID noises with other laws. If do not prefix “a” the parameter is estimated.

The exponential smoothing can also be applied to IID noises generated from other distributions. In the following, we propose to apply it to noises generated with **uniform, exponential, Poisson, and Student-t distributions**.

As we commented above, changing n doesn't affect the statistical properties of the IID noise, so below we will plot the estimated filter for different values of “a” while we keep $n = 100$.

The “a” values we will plot are: 0.01, 0.1, 0.2, 0.5, 0.8, 0.9.

- Uniform noise

```
# Uniform noise between -1 and 1

# First define the function to plot for varying values of "a"
hw_noise_plots_uniform <- function(n){

  set.seed(123)
  x <- runif(n, min = -1, max = 1)

  alphas <- c(0.01, 0.1, 0.2, 0.5, 0.8, 0.9)

  par(
    mfrow = c(2,3),
    oma = c(0,0,3,0), # outer margin for global title
    mar = c(4,4,2,1) # bottom, left, top, right margins
  )

  for(a in alphas){

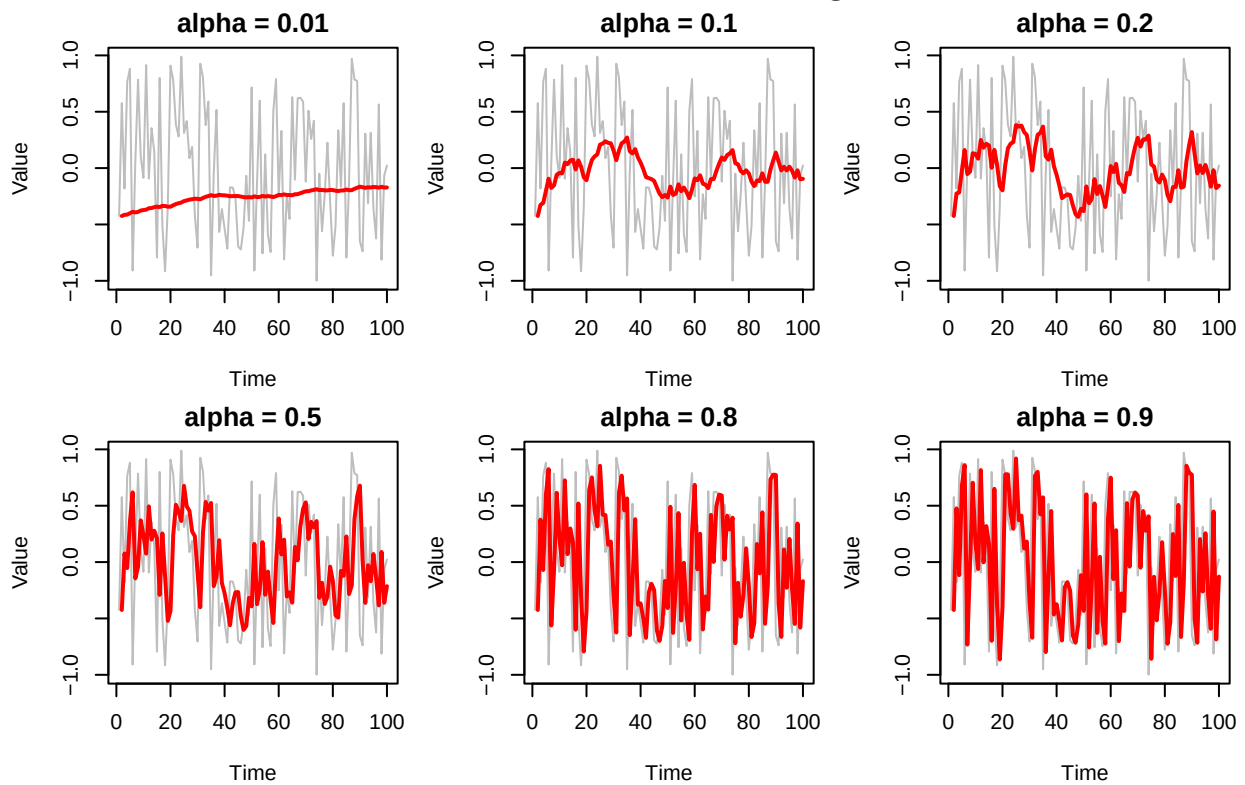
    y <- HoltWinters(x, alpha=a, beta=FALSE, gamma=FALSE)
    ylim <- range(x) # to fix same y-scale in the plot
    # plot noise
    plot(x,
         type="l",
         ylim = ylim,
         col="grey",
         main=paste("alpha =", a),
         xlab="Time",
         ylab="Value")

    # add smoothed signal
    lines(y$fitted[,1], col="red", lwd=2)
  }

  mtext(paste("Holt-Winters Uniform Noise filtering with n =", n),
        outer=TRUE, cex=1.3)
}

hw_noise_plots_uniform(100)
```

Holt-Winters Uniform Noise filtering with $n = 100$



- Exponential noise (positive skewed noise)

```
# Exponential noise (positive skewed noise)

# First define the function to plot for varying values of "a"
hw_noise_plots_exp <- function(n){

  set.seed(123)
  x <- rexp(n, rate = 1)

  alphas <- c(0.01, 0.1, 0.2, 0.5, 0.8, 0.9)

  par(
    mfrow = c(2,3),
    oma = c(0,0,3,0), # outer margin for global title
    mar = c(4,4,2,1)  # bottom, left, top, right margins
  )

  for(a in alphas){

    y <- HoltWinters(x, alpha=a, beta=FALSE, gamma=FALSE)
    ylim <- range(x) # to fix same y-scale in the plot
    # plot noise
    plot(x,
         ylim = ylim,
```

```

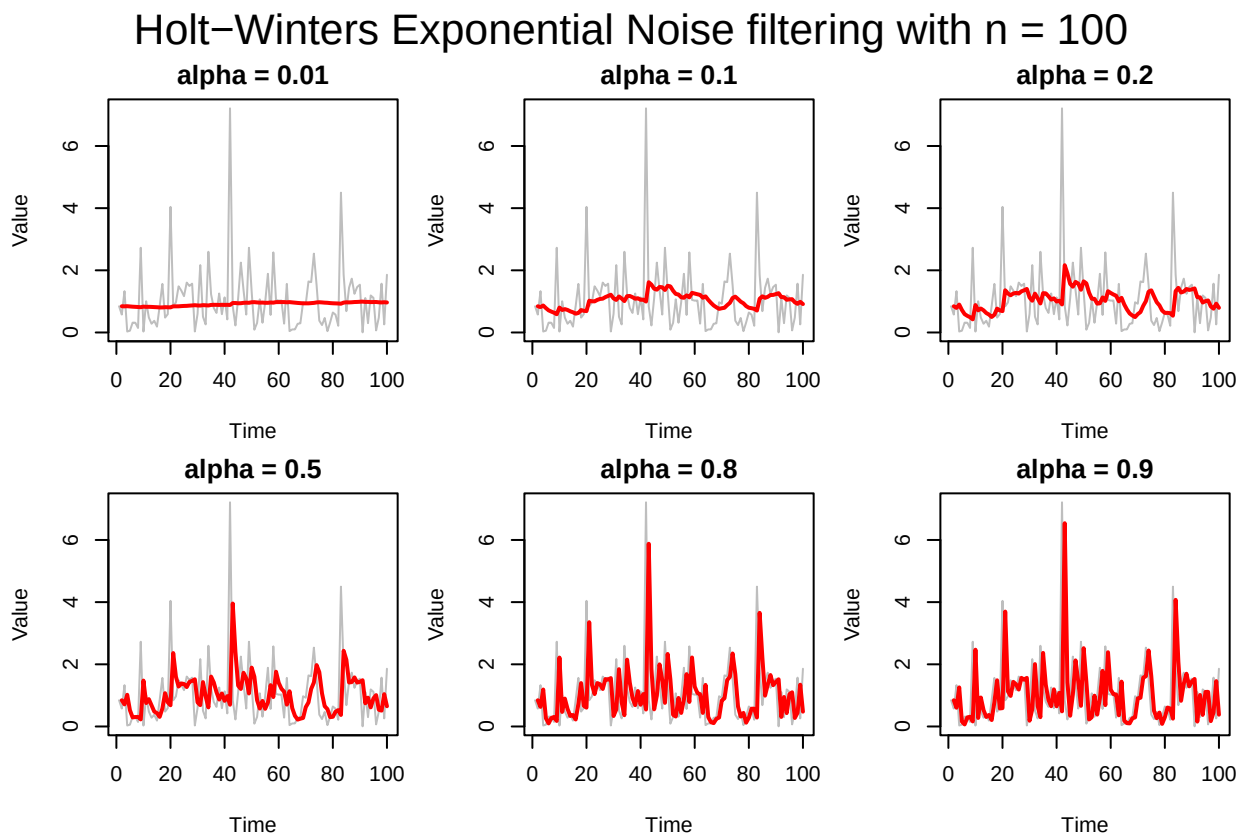
type="l",
col="grey",
main=paste("alpha =", a),
xlab="Time",
ylab="Value")

# add smoothed signal
lines(y$fitted[,1], col="red", lwd=2)
}

mtext(paste("Holt-Winters Exponential Noise filtering with n =", n),
      outer=TRUE, cex=1.3)
}

hw_noise_plots_exp(100)

```



- Poisson noise (discrete IID noise)

```

# Poisson noise

# First define the function to plot for varying values of "a"
hw_noise_plots_pois <- function(n){

  set.seed(123)
  x <- rpois(n, lambda = 3)

```

```

alphas <- c(0.01, 0.1, 0.2, 0.5, 0.8, 0.9)

par(
  mfrow = c(2,3),
  oma = c(0,0,3,0),  # outer margin for global title
  mar = c(4,4,2,1)   # bottom, left, top, right margins
)

for(a in alphas){

  y <- HoltWinters(x, alpha=a, beta=FALSE, gamma=FALSE)
  ylim <- range(x) # to fix same y-scale in the plot

  # plot noise
  plot(x,
    ylim = ylim,
    type="l",
    col="grey",
    main=paste("alpha =", a),
    xlab="Time",
    ylab="Value")

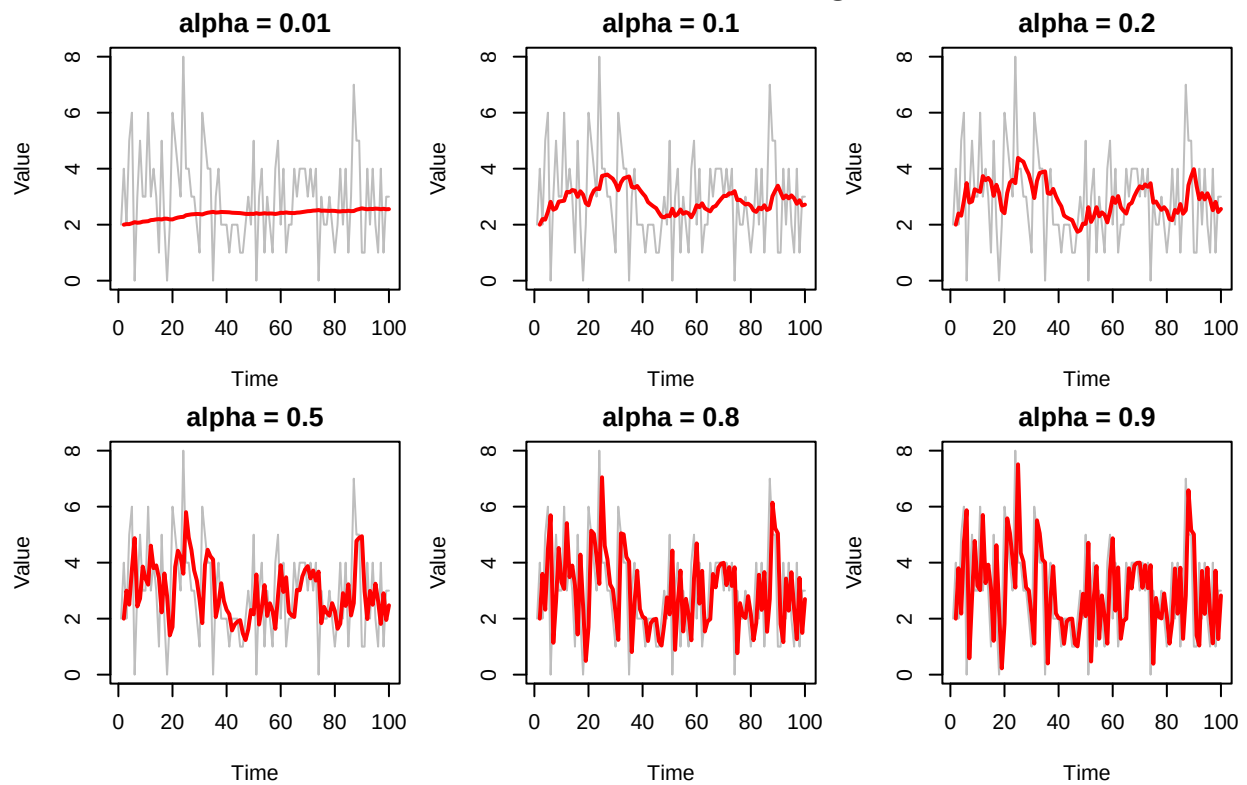
  # add smoothed signal
  lines(y$fitted[,1], col="red", lwd=2)
}

mtext(paste("Holt-Winters Poisson Noise filtering with n =", n),
      outer=TRUE, cex=1.3)
}

hw_noise_plots_pois(100)

```

Holt-Winters Poisson Noise filtering with $n = 100$



- Student-t noise

```
# Student-t noise

# First define the function to plot for varying values of "a"
hw_noise_plots_stud <- function(n){

  set.seed(123)
  x <- rt(n, df = 5)

  alphas <- c(0.01, 0.1, 0.2, 0.5, 0.8, 0.9)

  par(
    mfrow = c(2,3),
    oma = c(0,0,3,0), # outer margin for global title
    mar = c(4,4,2,1) # bottom, left, top, right margins
  )

  for(a in alphas){

    y <- HoltWinters(x, alpha=a, beta=FALSE, gamma=FALSE)
    ylim <- range(x) # to fix same y-scale in the plot

    # plot noise
    plot(x,
```

```

ylim = ylim,
type="l",
col="grey",
main=paste("alpha =", a),
xlab="Time",
ylab="Value")

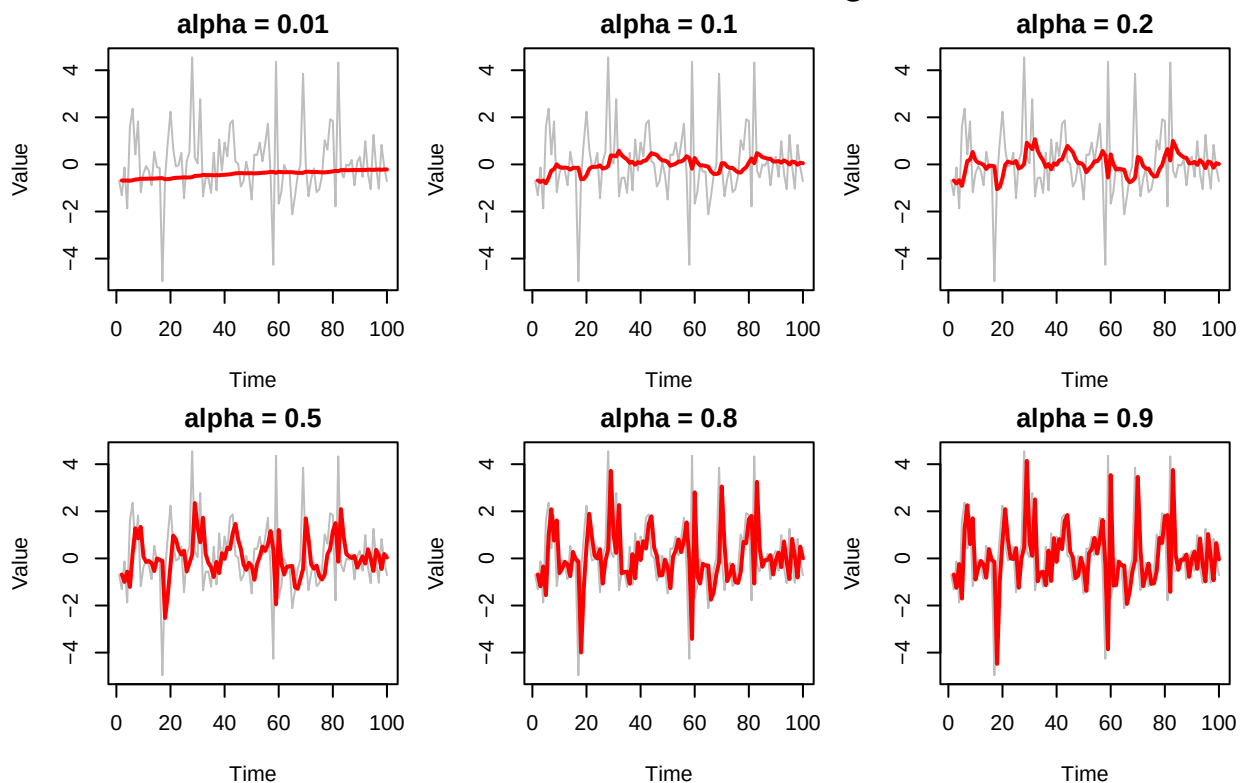
# add smoothed signal
lines(y$fitted[,1], col="red", lwd=2)
}

mtext(paste("Holt-Winters Student-t Noise filtering with n =", n),
      outer=TRUE, cex=1.3)
}

hw_noise_plots_stud(100)

```

Holt-Winters Student-t Noise filtering with n = 100



Observations:

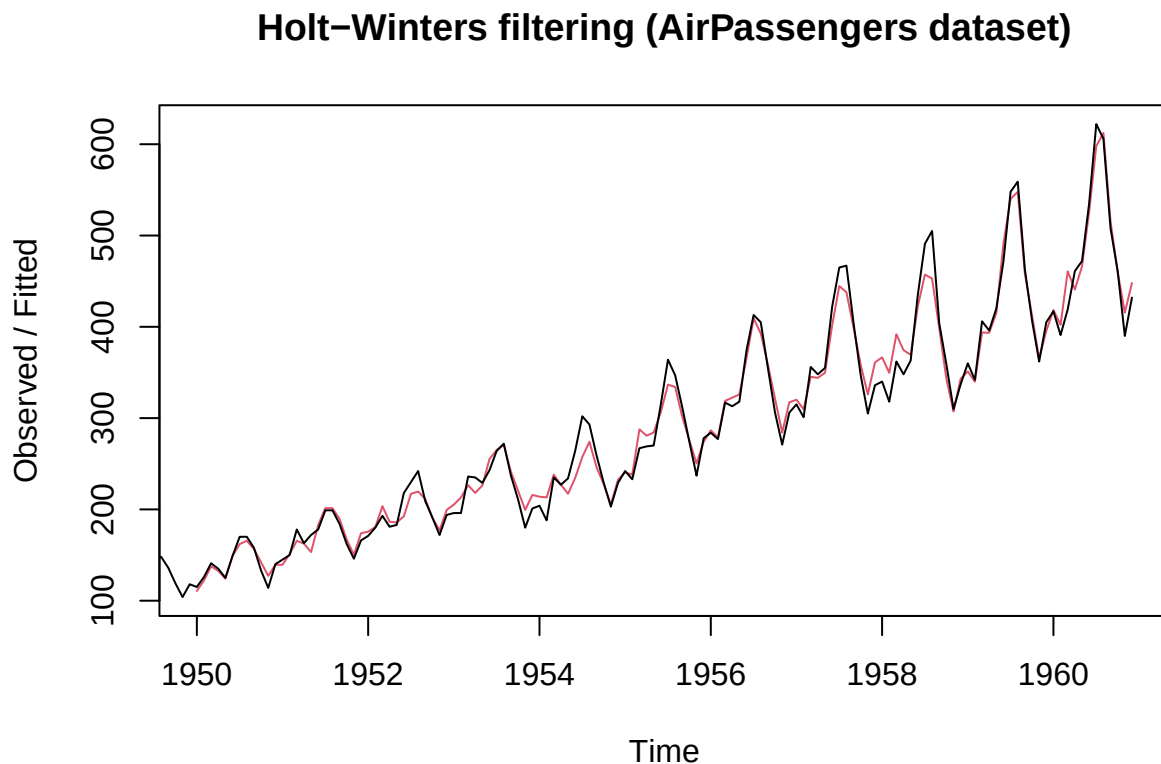
The general behavior of the filter remains similar:

- Small “a” (e.g. 0.01, 0.1) $\rightarrow$ more weight to past observations, very smooth estimate
- Large “a” (e.g. 0.8, 0.9) $\rightarrow$ more weight to recent observations, so the filtered series reacts quickly to changes in noise $\rightarrow$ noisy estimate

What changes with respect to `x <- rnorm(n)` is that the shape of the smoothed signal reflects the characteristics of the original distribution.

2. Apply Holt Winters smoothing to real data with trend and seasonality

```
x<-AirPassengers
a<-0.2
b<-0.3
c<-0.8
y<-HoltWinters(x,alpha=a,beta=b, gamma=c)
plot(y, main = "Holt-Winters filtering (AirPassengers dataset)")
```



The AirPassengers dataset shows a strong upward trend and a clear yearly seasonality.

The Holt-Winters method is therefore appropriate here, because it internally decomposes the series into level, trend, and seasonal components. The fitted values (red line) follow the upward trend over time and replicate the seasonal pattern, while applying a soft smoothing effect. In fact, the red curve fluctuates less than the original black series.

We will now see the role that the parameters “a”, “b”, “c” play in determining how quickly the model updates the level, trend, and seasonal components:

- “a” determines how quickly the **level** adapts to new observations.
With “a = 0.2” the level adapts gradually to new observations - while still giving substantial weight to the past ones.

- “b” determines how quickly the **trend** component adapts.
With “**b = 0.3**” the trend evolves smoothly over time while still responding to changes in the trend of the series. This results in a trend that is smoother than the original signal but still closely aligned with it.
- “c” determines how quickly the **seasonal** component is updated.
With “**c = 0.8**”, high weight is given to the recent seasonal observations, therefore the seasonal pattern adapts quickly to changes in seasonal amplitude.

As a result, the fitted curve closely follows the repeating yearly seasonal pattern while smoothing the fluctuations.

2.1 Change the series and use for example `log(x)`, `diff(log(x))`, `diff(x,12)`, etc.

The exercise suggests to now try to use the HoltWinters filter on transformations of the original signals, such as `log(x)`, `diff(log(x))`, `diff(x,12)`.

The transformations of the series can be applied in order to stabilize variance or remove trend and seasonality. For example:

- The logarithmic transformation reduces the growth of seasonal amplitude over time
- The first difference removes part of the trend
- The seasonal difference with lag 12 removes the yearly seasonal pattern

We will also use the HoltWinters on additional transformations:

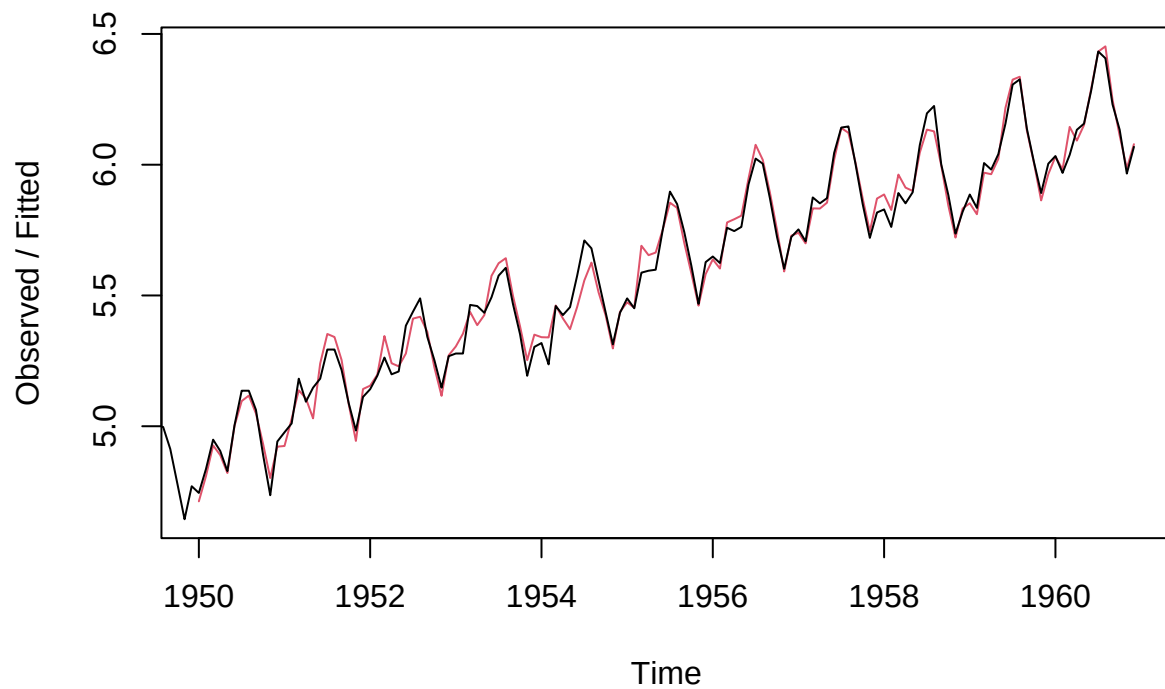
- `diff(log(AirPassengers), 12)` to combine the effect of the logarithmic transformation and seasonal difference with lag 12.
- `diff(diff(log(AirPassengers), 12)` to combine the effect of the first difference, with logarithmic transformation and seasonal difference with lag 12.

```
x0<-AirPassengers
a<-0.2
b<-0.3
c<-0.8

x1 <- log(x0)
x2 <- diff(log(x0))
x3 <- diff(x0, 12)
x4 <- diff(log(x0), 12)
x5 <- diff(diff(log(x0), 12))

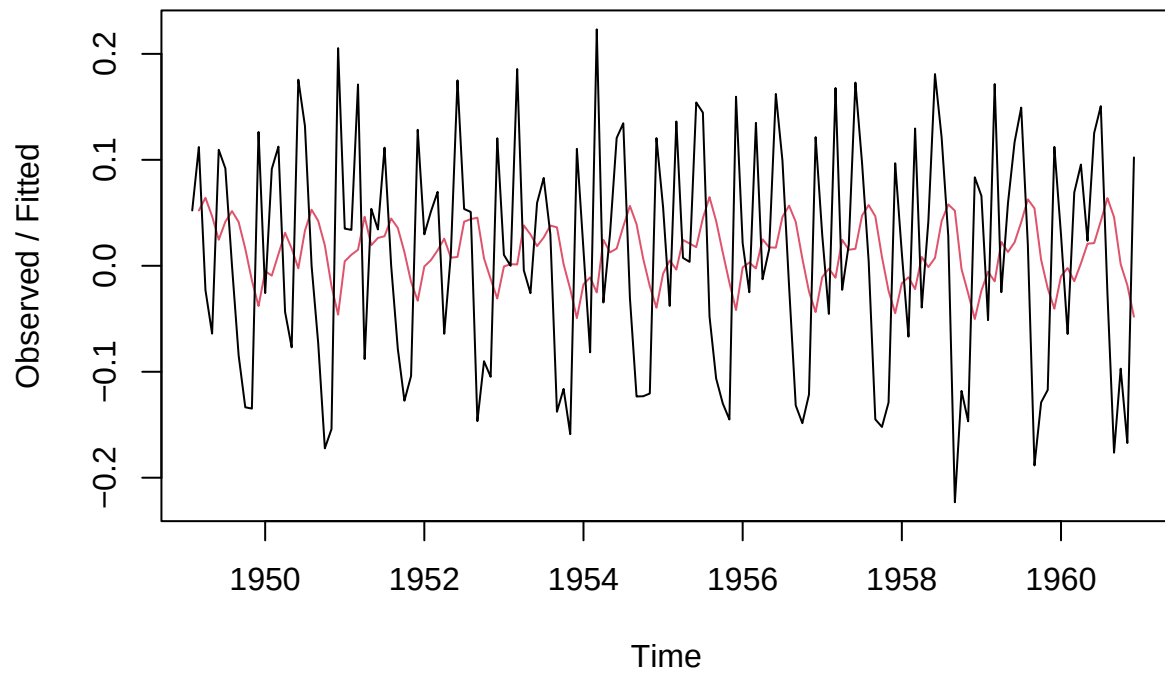
# Log(x)
y<-HoltWinters(x1,alpha=a,beta=b, gamma=c)
plot(y, main = "Holt-Winters filtering with log(AirPassengers)")
```

Holt-Winters filtering with $\log(\text{AirPassengers})$



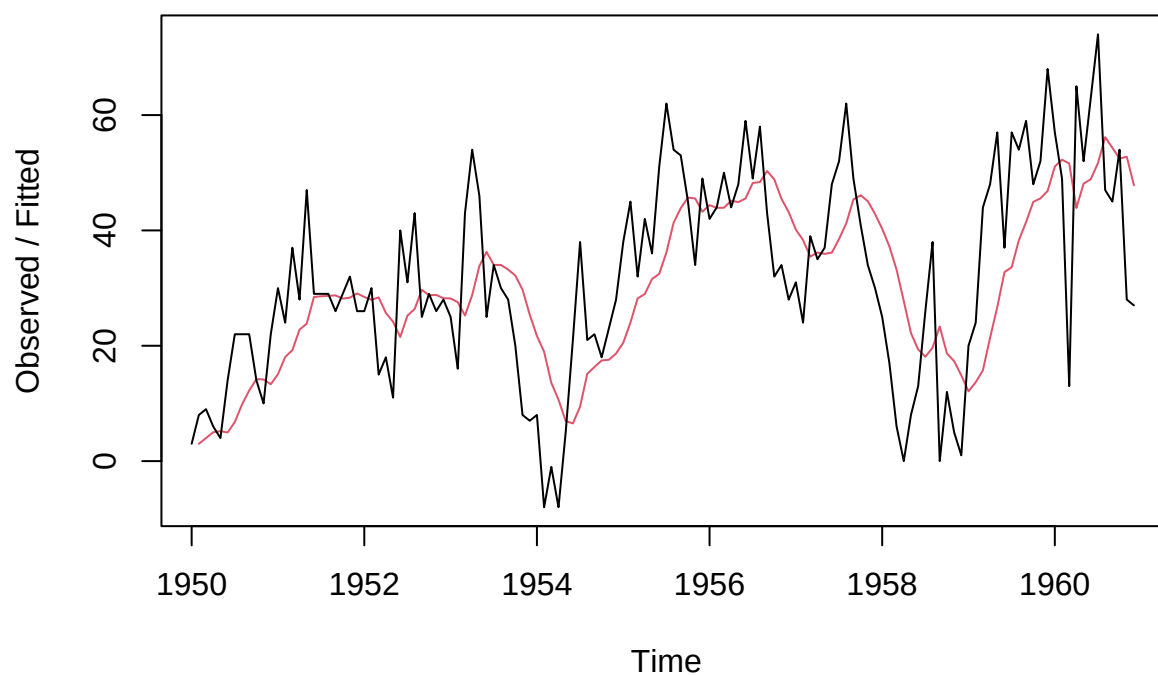
```
# For the differencing transformations, the trend and seasonality are already removed, so we will set b  
  
# diff(log(x))  
y<-HoltWinters(x2,alpha=a,beta=F, gamma=F)  
plot(y, main = "Holt-Winters filtering with diff(log(AirPassengers))")
```

Holt-Winters filtering with `diff(log(AirPassengers))`



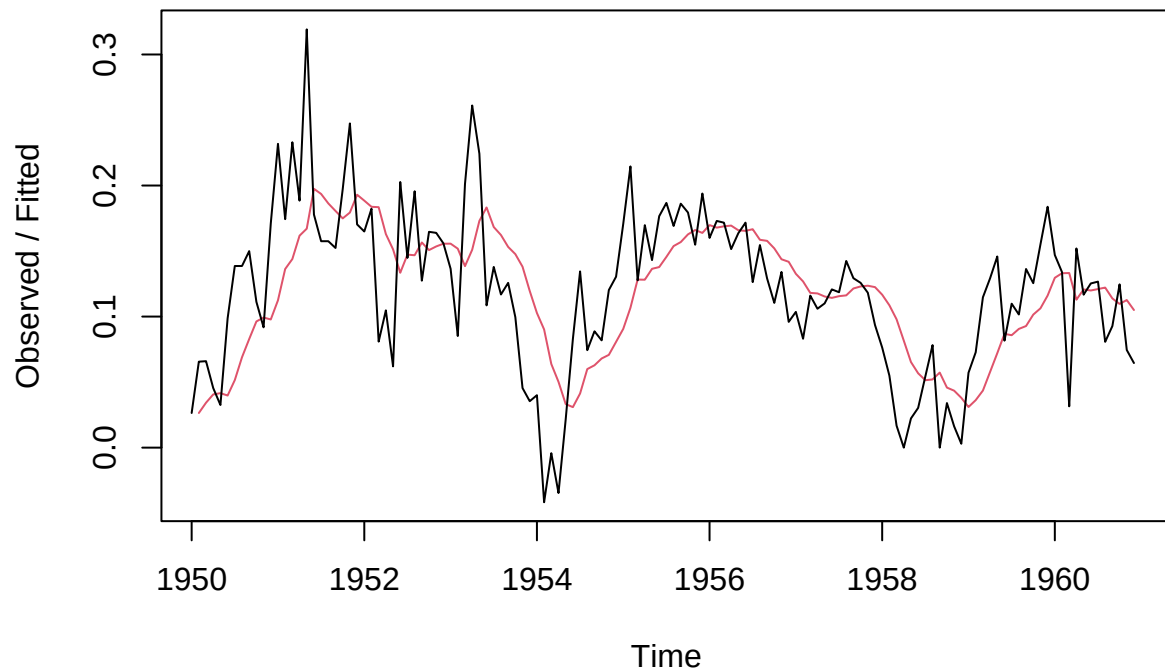
```
# diff(x,12)
y<-HoltWinters(x3,alpha=a,beta=F, gamma=F)
plot(y, main = "Holt-Winters filtering with diff(AirPassengers,12)")
```

Holt-Winters filtering with `diff(AirPassengers,12)`



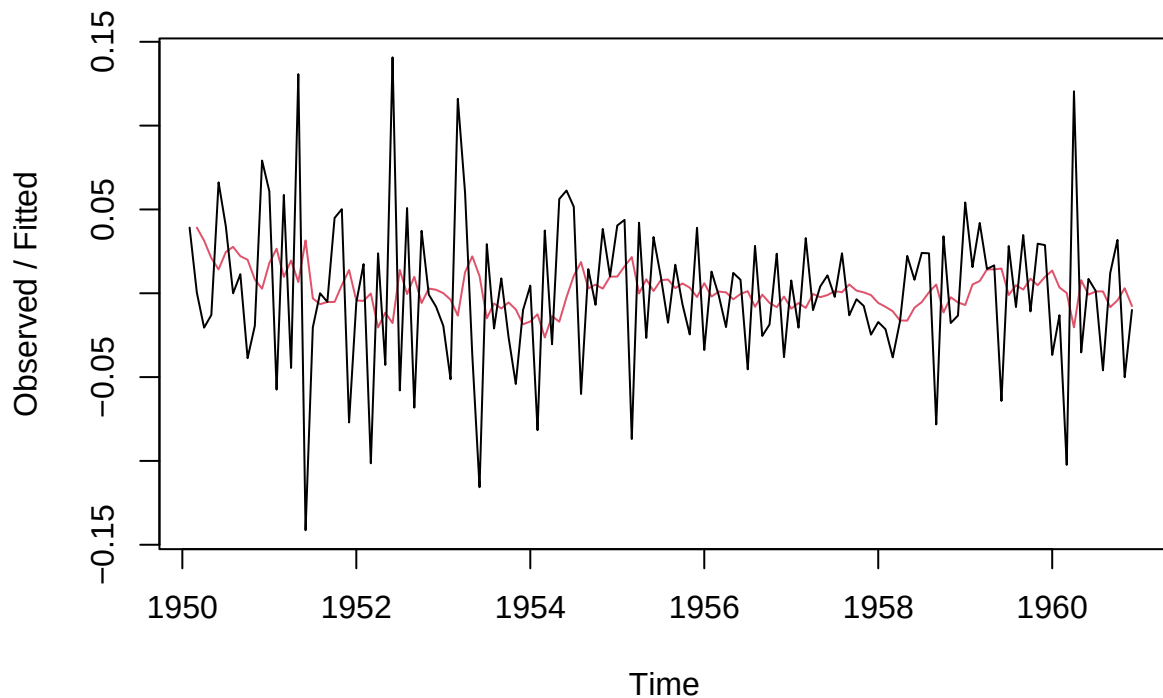
```
# diff(log(AirPassengers), 12)
y<-HoltWinters(x4,alpha=a,beta=F, gamma=F)
plot(y, main = "Holt-Winters filtering with diff(log(AirPassengers),12)")
```

Holt-Winters filtering with $\text{diff}(\log(\text{AirPassengers}), 12)$



```
# diff(diff(log(AirPassengers), 12)
y<-HoltWinters(x5,alpha=a,beta=F, gamma=F)
plot(y, main = "Holt-Winters filtering with diff(diff(log(AirPassengers), 12))")
```

Holt-Winters filtering with $\text{diff}(\text{diff}(\log(\text{AirPassengers}), 12))$



Observations:

1. **$\log(\text{AirPassengers})$**

While in the original series the seasonal peaks become larger over time (initial oscillations between 100 and 200, final oscillations between 400 and 600), in the log-transformed series the oscillations are more stable (initial oscillations between 0 and 5, final oscillations between 6 and 6.5 approximately). As a consequence, the Holt-Winters fitted curve therefore follows the trend and seasonal pattern more uniformly across the whole time range.

2. **$\text{diff}(\log(\text{AirPassengers}))$**

After applying the first difference to the log-transformed series, the trend component is removed. The series now oscillates around 0, and the Holt-Winters filter captures more the signal variations rather than the upward trend.

3. **$\text{diff}(\text{AirPassengers}, 12)$**

The seasonal difference with lag 12 tends to remove the yearly seasonal component. In fact, the resulting series no longer shows the repeating yearly peaks that were visible in the original data. The filtered series therefore highlights the remaining fluctuations.

4. **$\text{diff}(\log(\text{AirPassengers}), 12)$**

Here we combined the logarithmic transformation with the seasonal difference: so we stabilized the variance and removed the yearly seasonal pattern at the same time.

The resulting series appears more stationary, fluctuating between 0 and 0.3 approximately.

The Holt-Winters filter therefore produces a smoother signal that fluctuates around 0.1, capturing the

remaining variations in the series.

5. `diff(diff(log(AirPassengers), 12))`

In this case we applied an additional first difference to the transformation of point 4. This further removes any remaining trend component. So the idea was to see if in the signal of point 4 there was any remaining trend component.

The plots of point 4 and 5 indeed show a different resulting series: the first difference produced a more stationary signal, with fluctuations centered around zero.

In this case the Holt-Winters filter smoothens the fluctuations rather than modeling trend or seasonality, and it captures the few remaining variations in the series.

2.2 Change the values of parameters a, b and c

For every combination of a, b and c, we will plot the Holt-Winters filtering on the above transformations 1, 2, 3, 4, 5, 6.

To avoid repeating the code every time, we first create a function that takes as input a, b and c. We will plot in the first line the `AirPassengers` and `Log(AirPassengers)` with the same scale; and in the other lines the other 4 transformations with a same scale between them. In this way we can easily see the changes between transformations. We won't keep the same scale between all the 6 plots because `AirPassengers` and `Log(AirPassengers)` have much larger scale, so the other 4 plots would be visible as lines (all information of the visualization would be lost).

```
plot_hw_transforms <- function(a, b, c) {  
  
  # Series  
  x1 <- AirPassengers  
  x2 <- log(x1)  
  x3 <- diff(log(x1))  
  x4 <- diff(x1, 12)  
  x5 <- diff(log(x1), 12)  
  x6 <- diff(diff(log(x1), 12))  
  
  # Holt-Winters fits  
  y1 <- HoltWinters(x1, alpha = a, beta = b, gamma = c)  
  y2 <- HoltWinters(x2, alpha = a, beta = b, gamma = c)  
  y3 <- HoltWinters(x3, alpha = a, beta = b, gamma = c)  
  y4 <- HoltWinters(x4, alpha = a, beta = b, gamma = c)  
  y5 <- HoltWinters(x5, alpha = a, beta = b, gamma = c)  
  y6 <- HoltWinters(x6, alpha = a, beta = b, gamma = c)  
  
  # Shared scale for large-magnitude series  
  ylim_big <- range(  
    c(x1, x4, y1$fitted[,1], y4$fitted[,1]),  
    na.rm = TRUE  
  )  
  
  # Shared scale for log-based transformed series  
  ylim_small <- range(  
    c(x2, x3, x5, x6,  
      y2$fitted[,1], y3$fitted[,1], y5$fitted[,1], y6$fitted[,1]),  
    na.rm = TRUE  
  )  
}
```



```

# Layout: 2 plots on top, 4 below
layout(matrix(c(1,2,
                3,4,
                5,6), nrow = 3, byrow = TRUE))

par(oma = c(0, 0, 3, 0), mar = c(4, 4, 3, 1))

plot_hw <- function(x, y, title, ylim) {
  plot(x, col = "grey60", main = title, ylim = ylim)
  lines(y$fitted[,1], col = "red", lwd = 2)
}

# Top row
plot_hw(x1, y1, "AirPassengers", ylim_big)
plot_hw(x4, y4, "diff(AirPassengers, 12)", ylim_big)

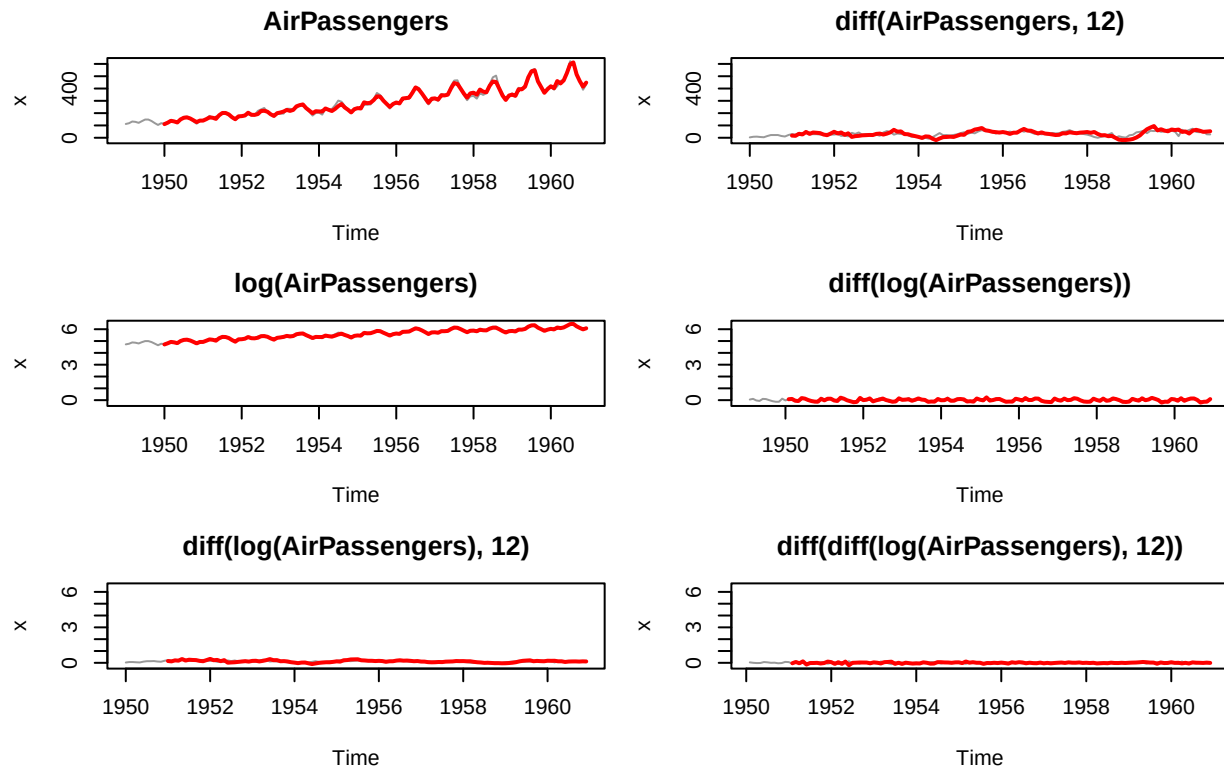
# Bottom 4
plot_hw(x2, y2, "log(AirPassengers)", ylim_small)
plot_hw(x3, y3, "diff(log(AirPassengers))", ylim_small)
plot_hw(x5, y5, "diff(log(AirPassengers), 12)", ylim_small)
plot_hw(x6, y6, "diff(diff(log(AirPassengers), 12))", ylim_small)

mtext(paste("a =", a, ", b =", b, ", c =", c), outer = TRUE, cex = 1.3)
}

# Initial values of a, b, c
plot_hw_transforms(0.2, 0.3, 0.8)

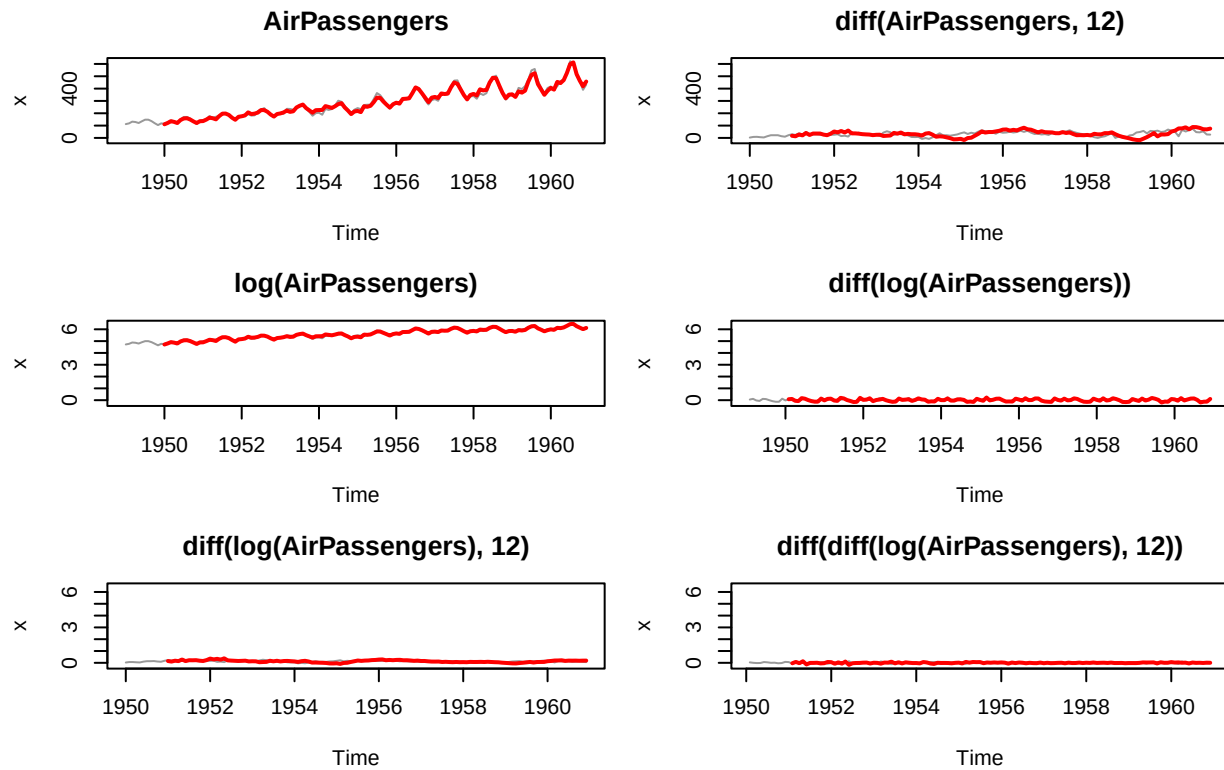
```

$a = 0.2$, $b = 0.3$, $c = 0.8$



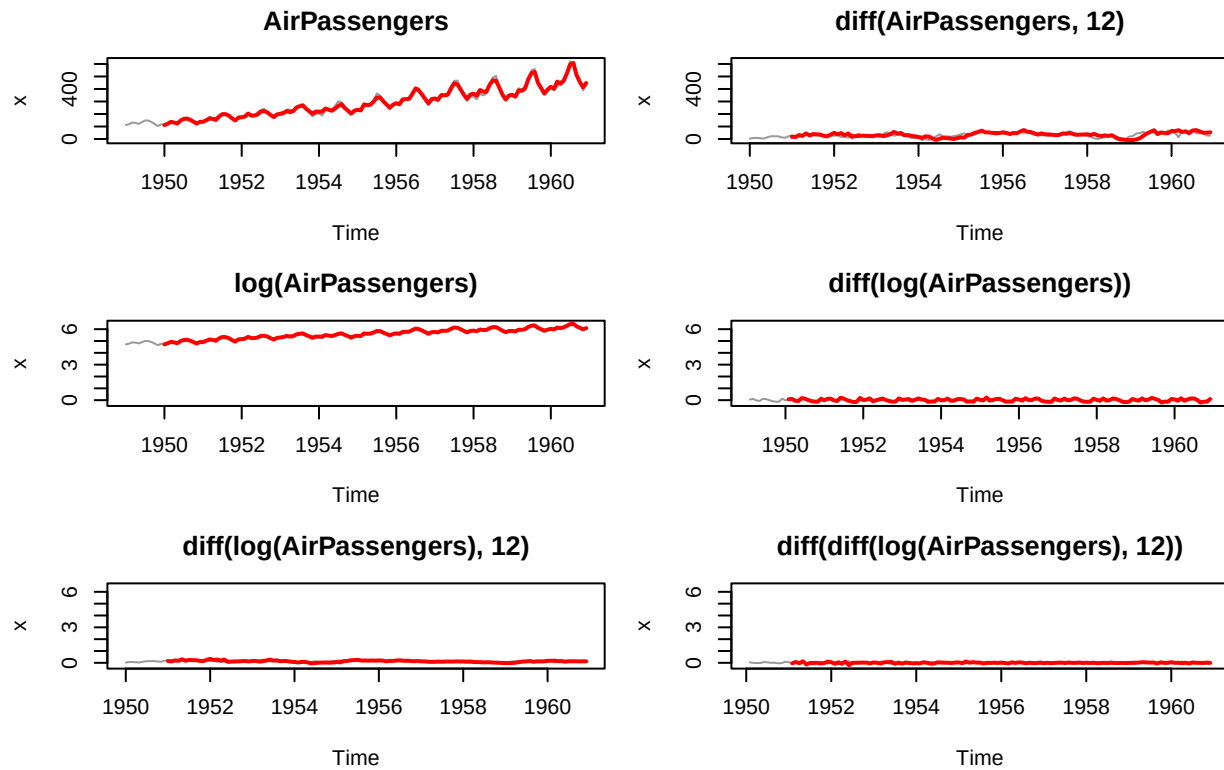
```
# Decrease only a (a was 0.2)
plot_hw_transforms(0.05, 0.3, 0.8)
```

$a = 0.05$, $b = 0.3$, $c = 0.8$



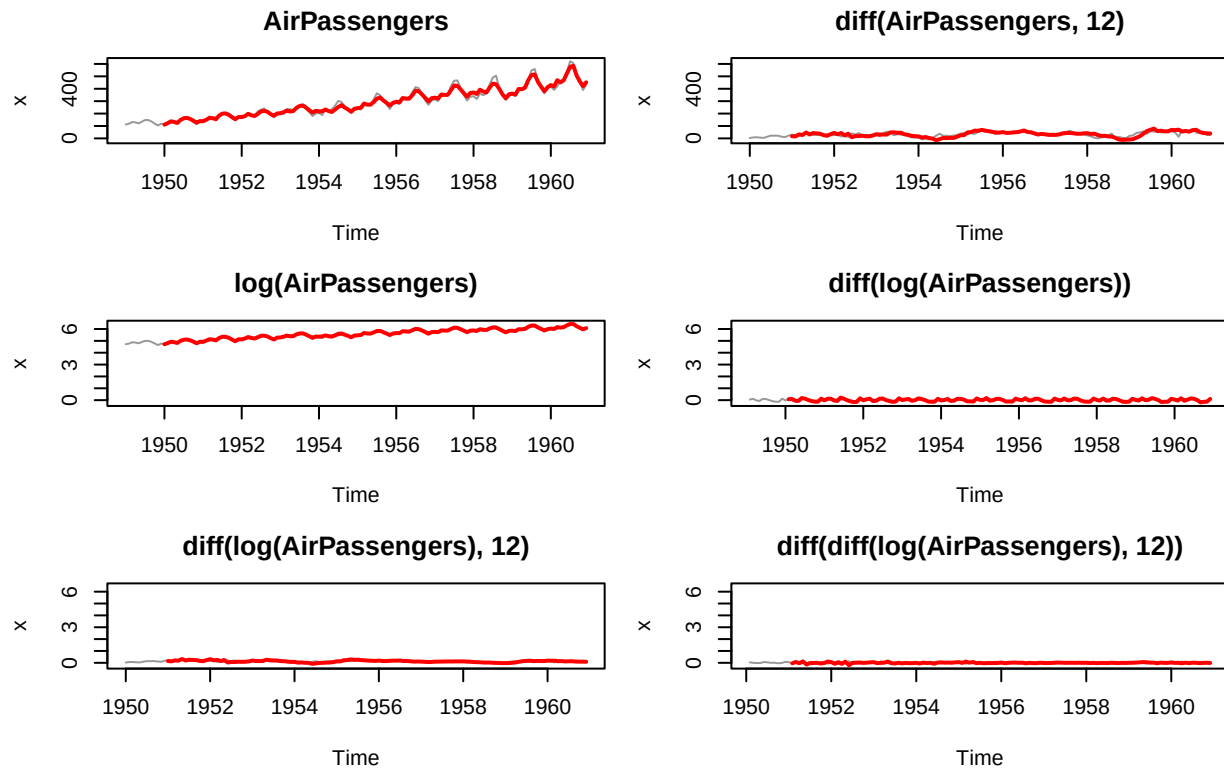
```
# Decrease only b (b was 0.3)
plot_hw_transforms(0.2, 0.1, 0.8)
```

$a = 0.2$, $b = 0.1$, $c = 0.8$



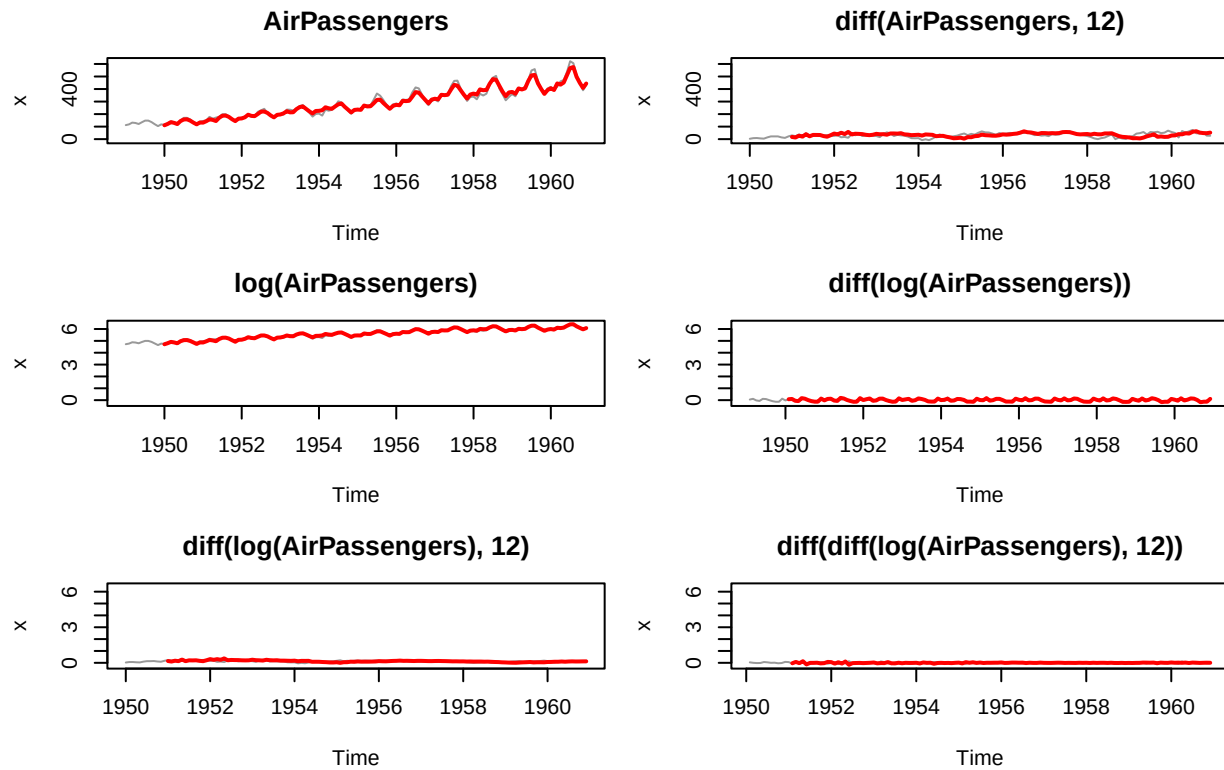
```
# Decrease only c (c was 0.8)
plot_hw_transforms(0.2, 0.3, 0.4)
```

$a = 0.2$, $b = 0.3$, $c = 0.4$



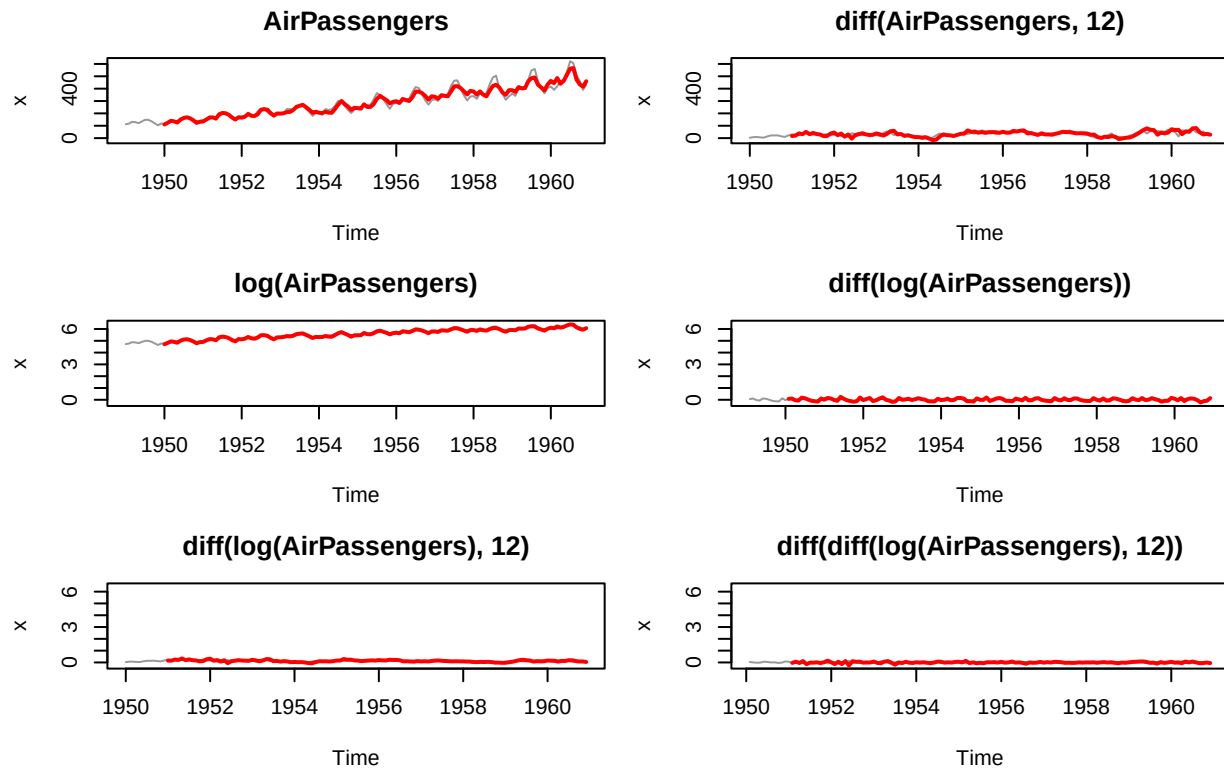
```
# Decrease a, b, c  
plot_hw_transforms(0.05, 0.1, 0.4)
```

$a = 0.05$, $b = 0.1$, $c = 0.4$



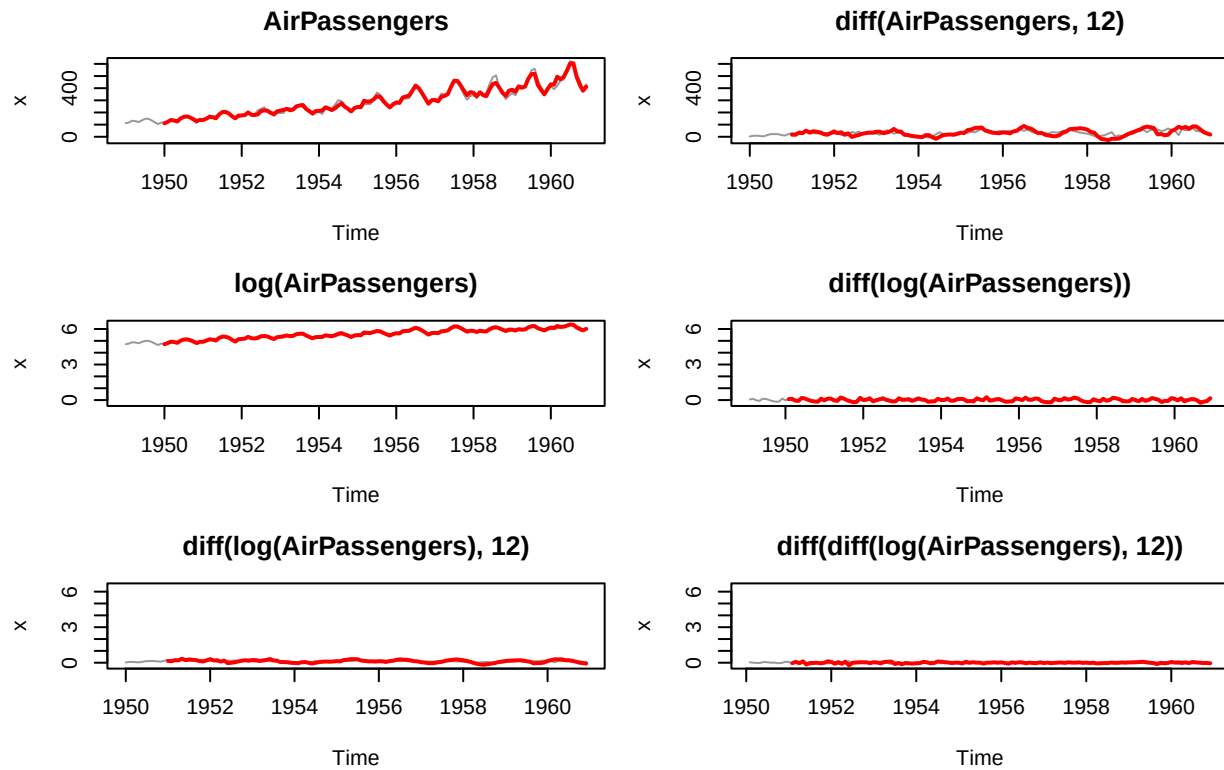
```
# Increase only a  
plot_hw_transforms(0.5, 0.3, 0.8)
```

$a = 0.5$, $b = 0.3$, $c = 0.8$



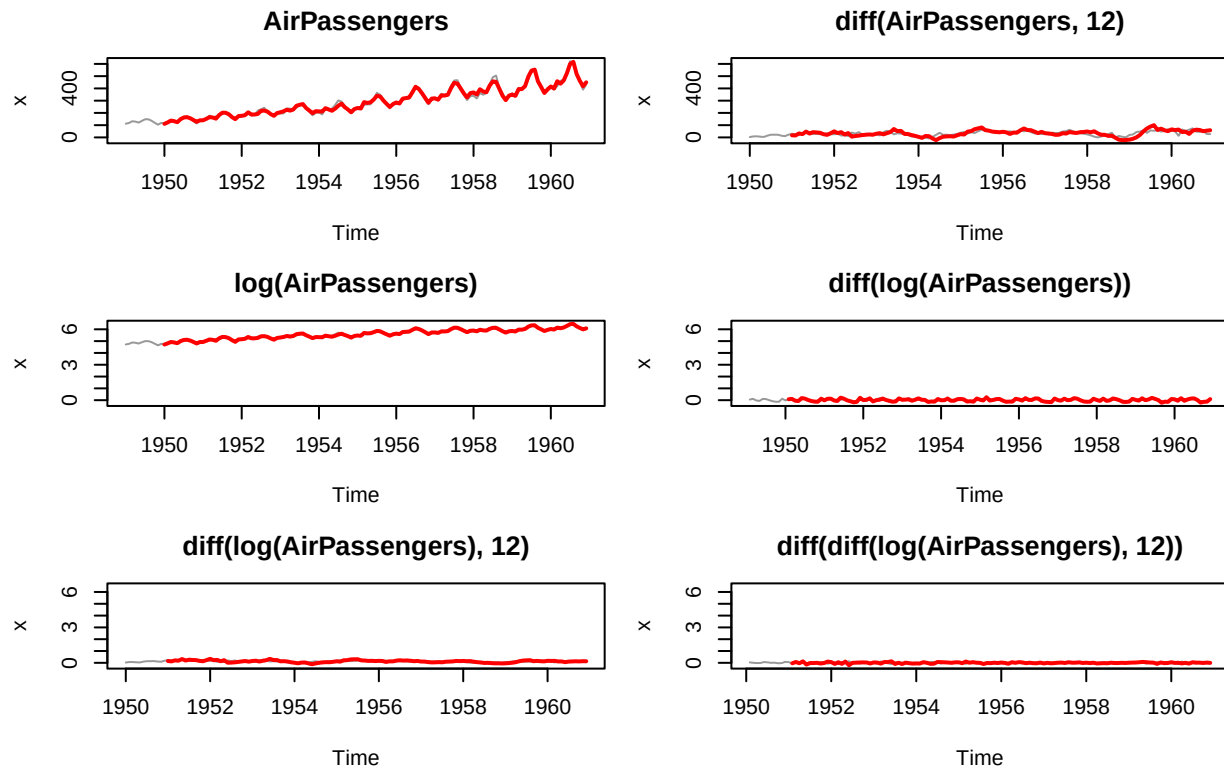
```
# Increase only b  
plot_hw_transforms(0.2, 0.6, 0.8)
```

$a = 0.2$, $b = 0.6$, $c = 0.8$



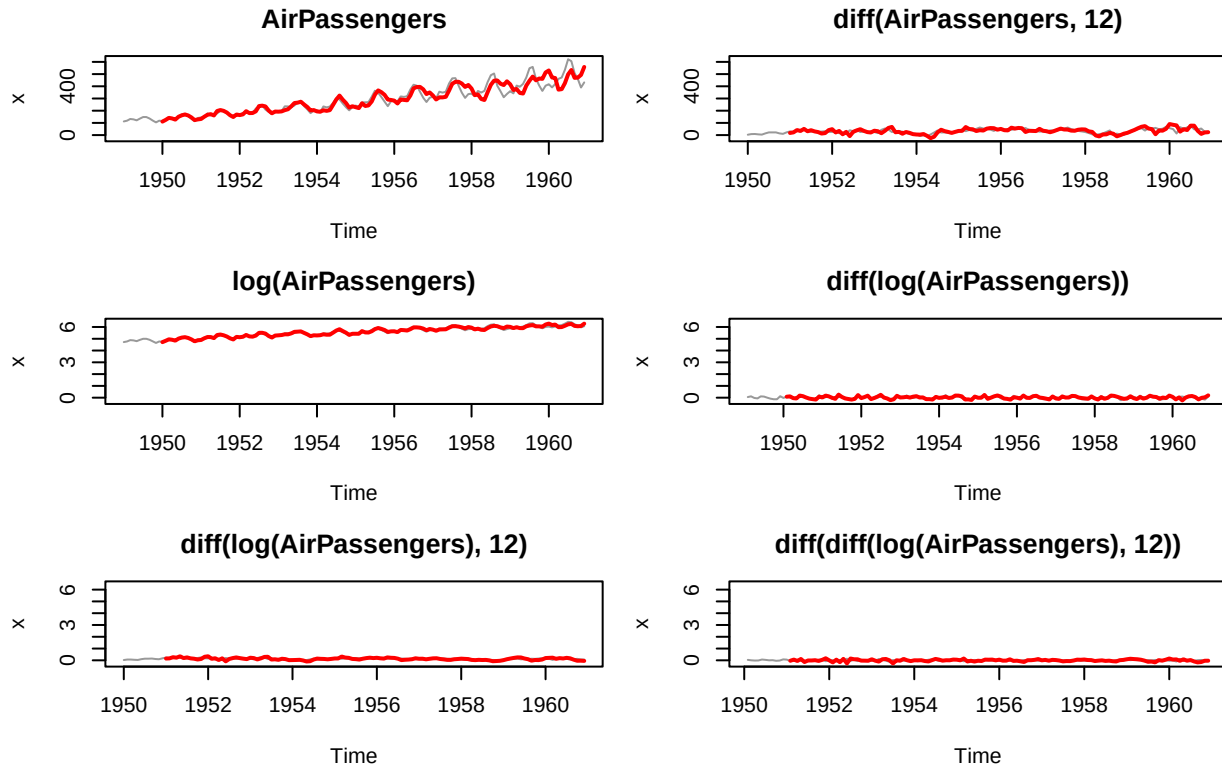
```
# Increase only c  
plot_hw_transforms(0.2, 0.3, 0.9)
```


$a = 0.2$, $b = 0.3$, $c = 0.9$



```
# Increase a, b, c  
plot_hw_transforms(0.5, 0.6, 0.9)
```

$$a = 0.5, b = 0.6, c = 0.9$$



Since the original series and the transformed series have very different magnitudes, using a single common scale for all subplots makes the transformed signals hard to interpret. Therefore, we decided to use a common scale between AirPassengers and Log(AirPassengers) and a common scale between the other transformations.

As we saw before, “a” controls the smoothing of the level, “b” of the trend, “c” of the seasonality.

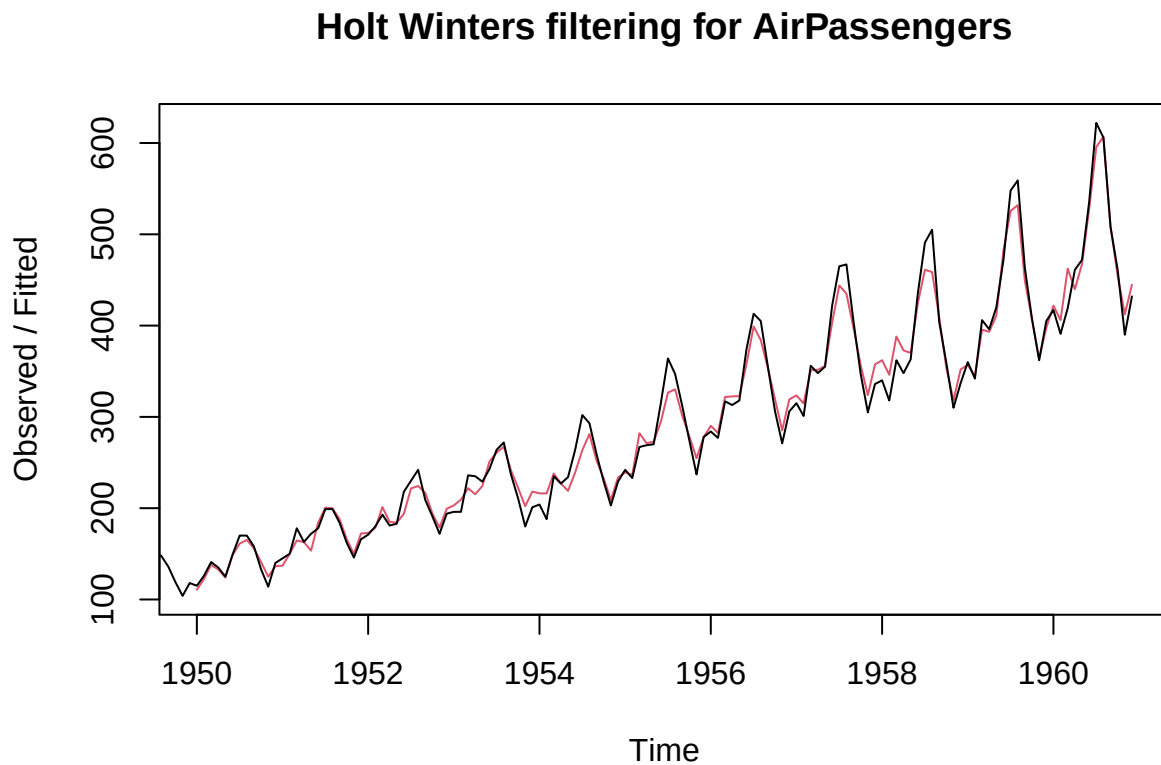
So in the plots above we tried to first decrease/increase each parameter individually, and then all of them together:

- **Decrease (increase) only a → plot 2 (plot 6)**
The level reacts slowly (quickly) to new observations. The fitted curve follows the data more gradually (closely) and adapts slowly (rapidly) to changes in the series.
- **Decrease (increase) only b → plot 3 (plot 7)**
The trend component changes more slowly (faster) over time: the estimated trend becomes smoother and less sensitive to variations in the data (adjusts faster to changes in the slope of the series).
- **Decrease (increase) only c → plot 4 (plot 8)**
The seasonal component becomes more stable (adapts more quickly to recent seasonal patterns). The seasonal pattern changes less over time (becomes more reactive to new observations) and the model gives more weight to past (recent) seasonal estimates.
- **Decrease (increase) a, b, c → plot 5 (plot 9)**
When all parameters decrease (increase), the model reacts slowly (faster) to new information. The fitted curve adapts gradually (quickly) to changes in level, trend, and seasonality.

3. Holt Winters prediction

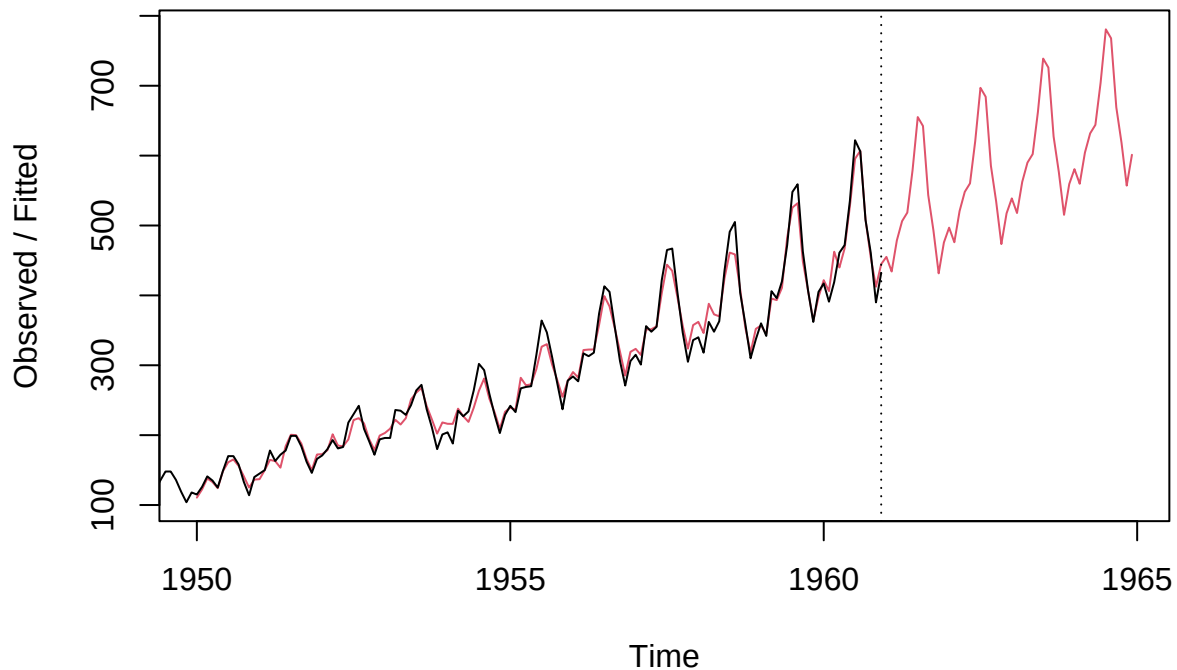
Now we will see that the Holt-Winters method can also be used to forecast future observations.

```
x<-AirPassengers
a<-0.3
b<-0.1
c<-0.8
y<-HoltWinters(x,alpha=a,beta=b,gamma=c)
plot(y, main = "Holt Winters filtering for AirPassengers")
```



```
z<-predict(y,n.ahead=4*12)
plot(y,z,lty=1:2, main = "Holt Winters filtering and prediction for AirPassengers")
```

Holt Winters filtering and prediction for AirPassengers



As we can see in the plot above, the red line represents:

- on the left of the dashed line $\rightarrow$ filter on the observed time series
- on the right of the dashed line $\rightarrow$ prediction for the next 4 years (`predict(y,n.ahead=4*12)`)

The forecast shows a continuation of the upward trend together with the repeating yearly seasonal pattern. Also, the amplitude of the seasonal fluctuations keeps increasing as the level of the series grows.

4. “a”, “b”, “c” are not prefixed

In this case we can create the Holt-Winters filter without giving explicit values of a, b and c. They will be automatically estimated, so we will use the commands “y\$alpha”, “y\$beta”, “y\$gamma” to read the estimated values.

```
x<-AirPassengers
y<-HoltWinters(x)
y$alpha
```

```
##      alpha
## 0.2479595
```

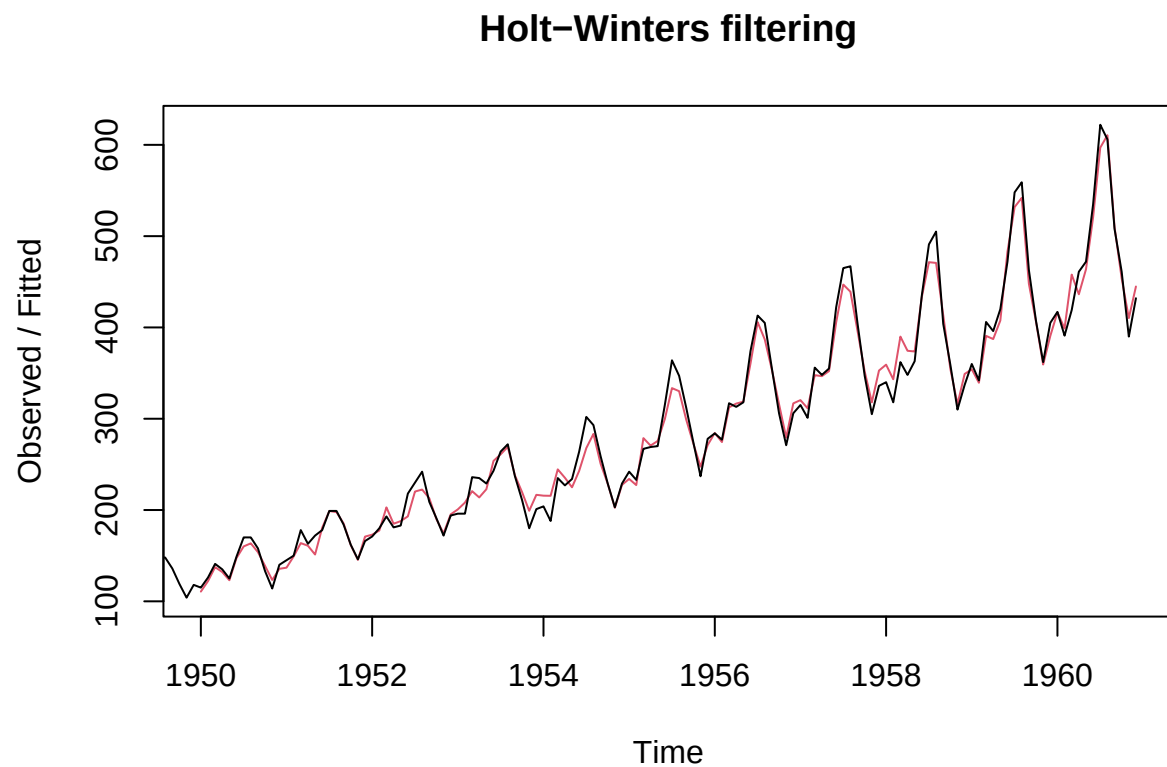
```
y$beta
```

```
##      beta
## 0.03453373
```

```
y$gamma
```

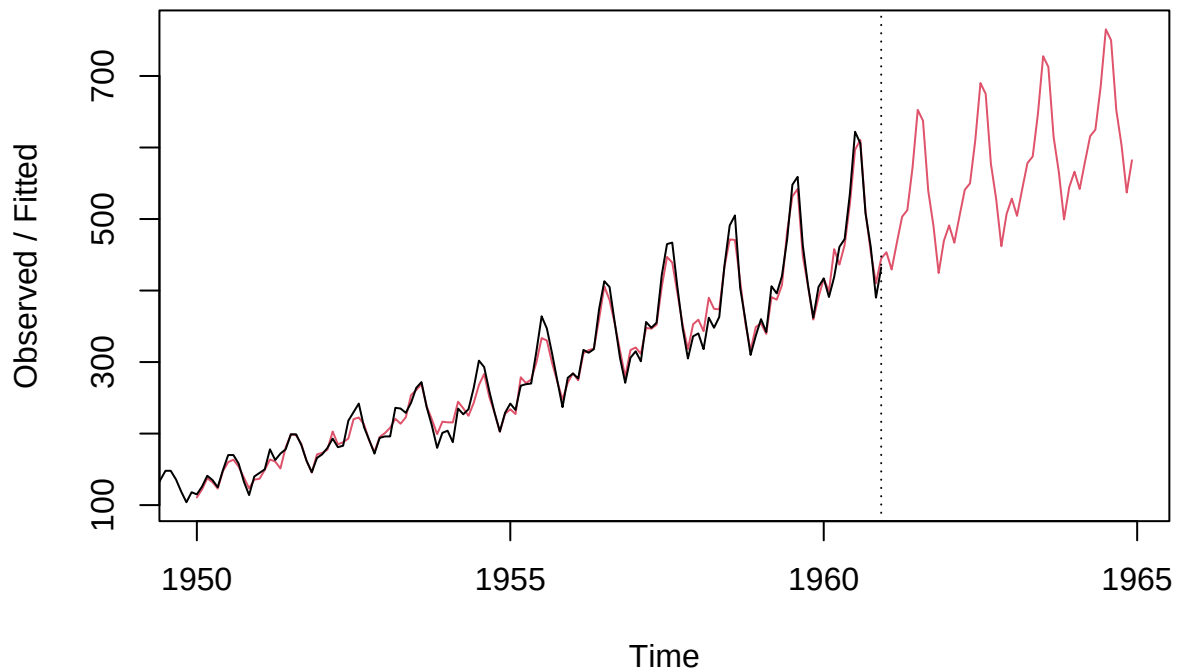
```
## gamma
##      1
```

```
plot(y)
```



```
z<-predict(y,n.ahead=4*12)
plot(y,z,lty=1:2)
```

Holt-Winters filtering



In order to minimize the forecast error, and to provide an optimal balance between responsiveness to new observations and stability of the level, trend, and seasonal components, the Holt-Winters model estimated the optimal parameters to be:

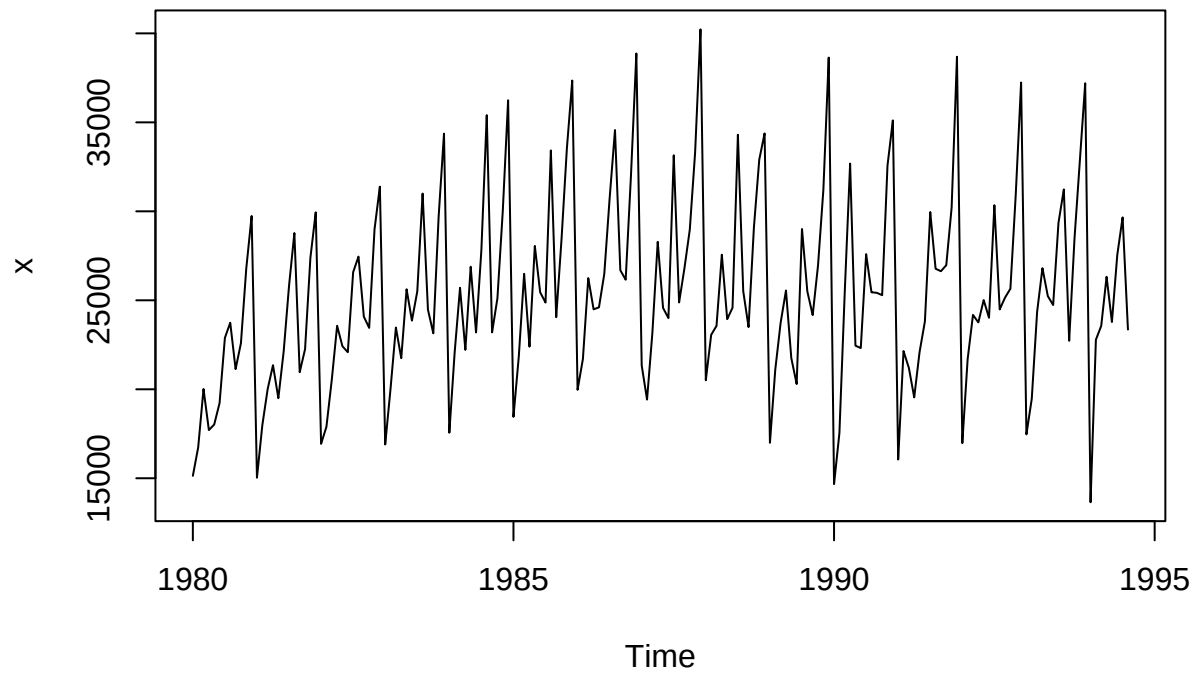
- $\alpha = 0.2479595$
- $\beta = 0.03453373$
- $\gamma = 1$

5. Wine sales in Australia

We will now see the effect of the Holt-Winters filtering and prediction on different data sets.

```
x<-wineind
y<-HoltWinters(x)
plot(x, main = "wineind dataset")
```

wineind dataset



```
y$alpha
```

```
##      alpha  
## 0.01655901
```

```
y$beta
```

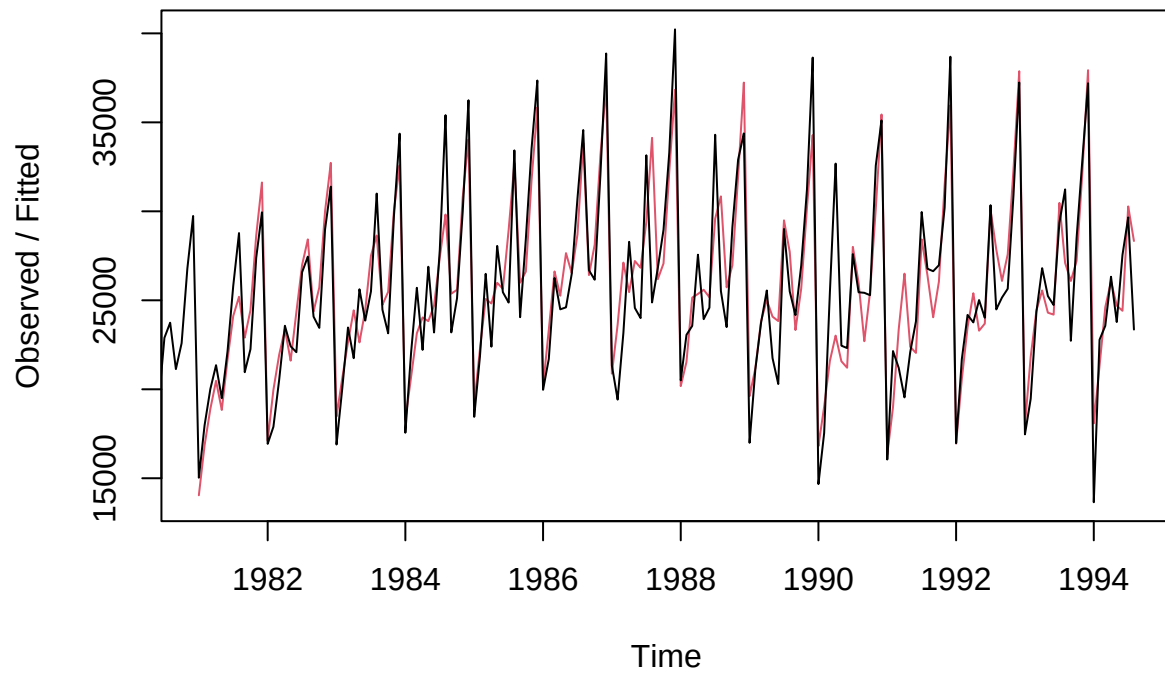
```
##      beta  
## 0.8125719
```

```
y$gamma
```

```
##      gamma  
## 0.2759254
```

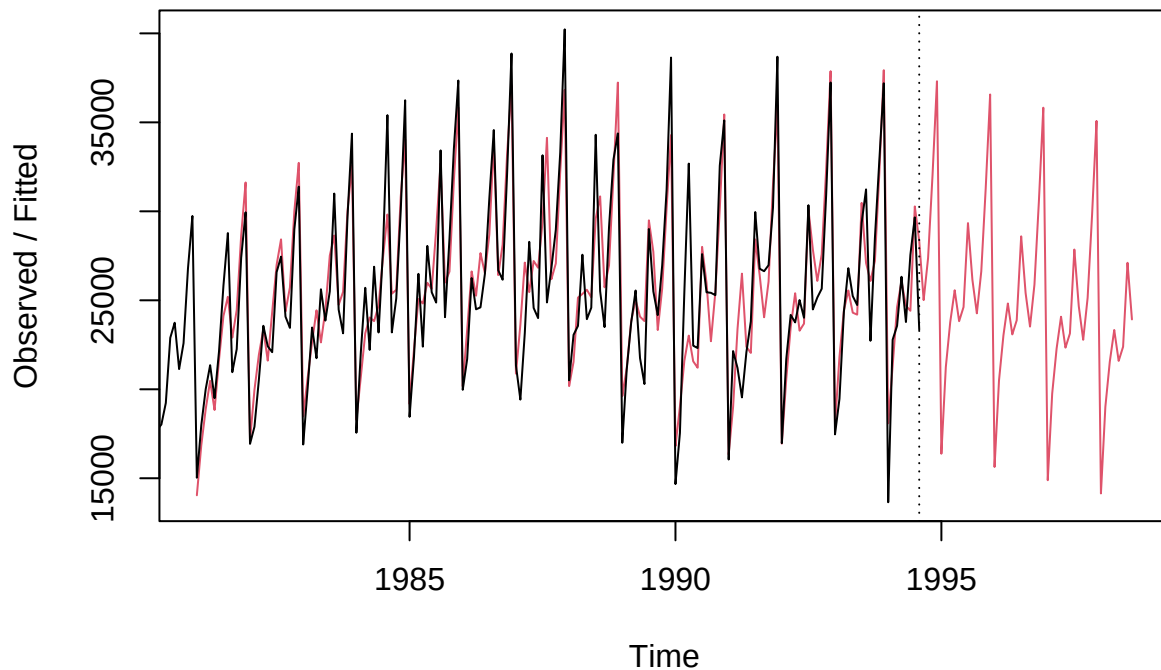
```
plot(y)
```

Holt-Winters filtering



```
z<-predict(y,n.ahead=4*12)
plot(y,z,lty=1:2, main = "Holt-Winters filtering and prediction")
```


Holt-Winters filtering and prediction



Observations:

In this case, the Holt-Winters method automatically estimates the parameters as:

- $\alpha = 0.01655901$
- $\beta = 0.812572$
- $\gamma = 0.2759254$

Automatic parameters selection:

“a” is very small, meaning that the level component changes slowly and gives strong weight to past observations. “b” is large (close to 1), which makes the model adapt quickly to changes in the trend. “c” takes a moderate value (around 0.27), meaning that the seasonal pattern is updated gradually over time rather than reacting too strongly to the fluctuations.

Forecast:

The forecast continues the seasonal fluctuations with approximately the same amplitude, indicating that the model expects the seasonal structure of the series to remain stable in the future.

5.1 other examples

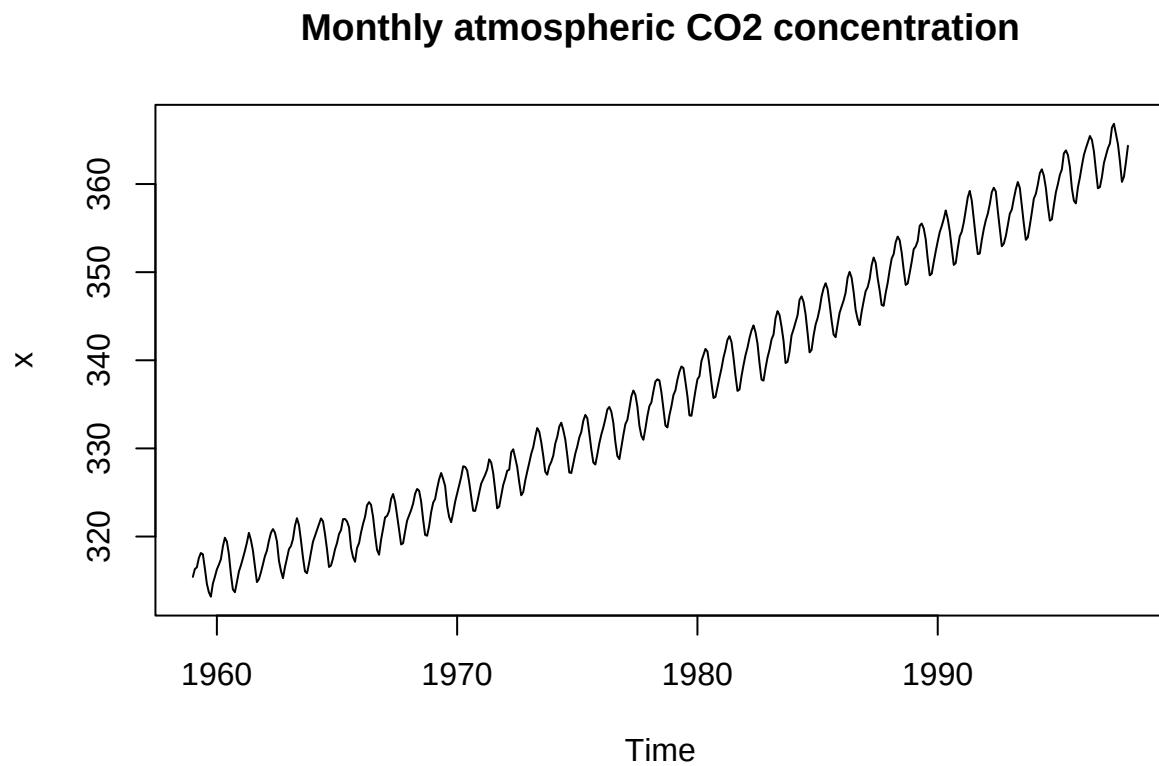
We will now try Holt-Winters on more data sets:

- Monthly atmospheric CO concentration
- UK gas consumption
- Monthly temperature - Nottingham

```
# Monthly atmospheric CO concentration

x <- co2
y <- HoltWinters(x)

plot(x, main = "Monthly atmospheric CO2 concentration")
```



```
y$alpha
```

```
##      alpha
## 0.5126484
```

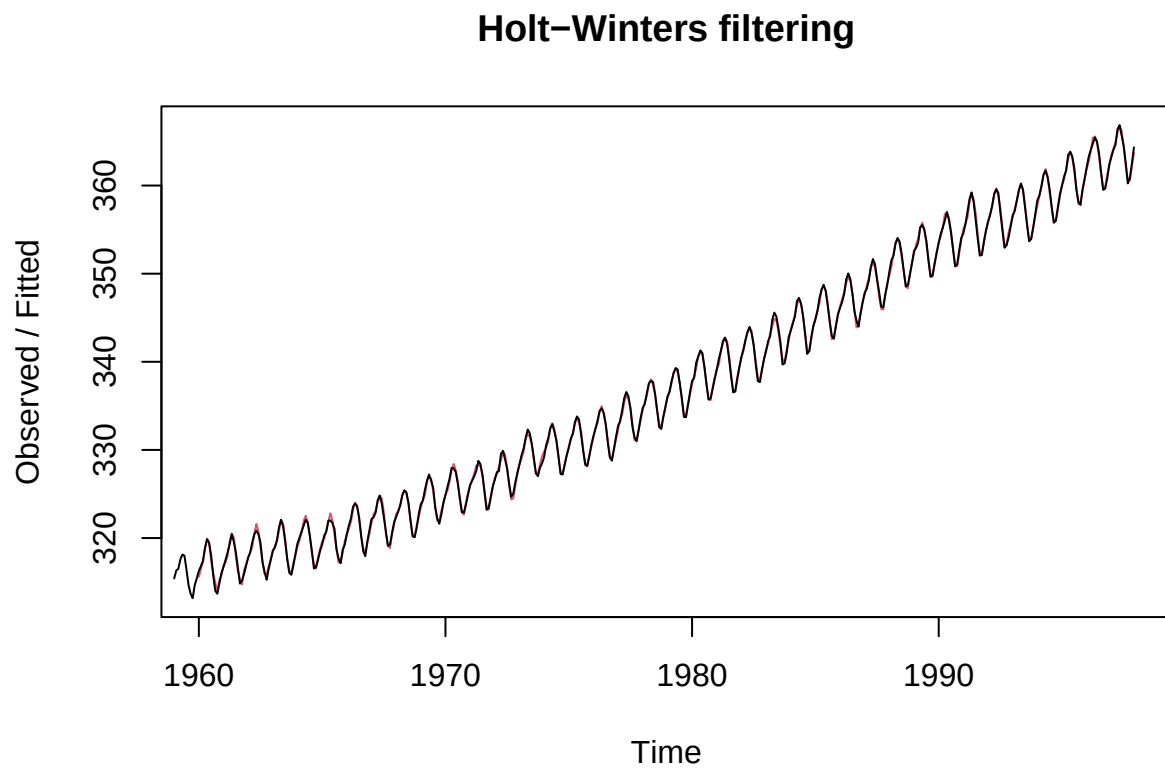
```
y$beta
```

```
##      beta
## 0.009497669
```

```
y$gamma
```

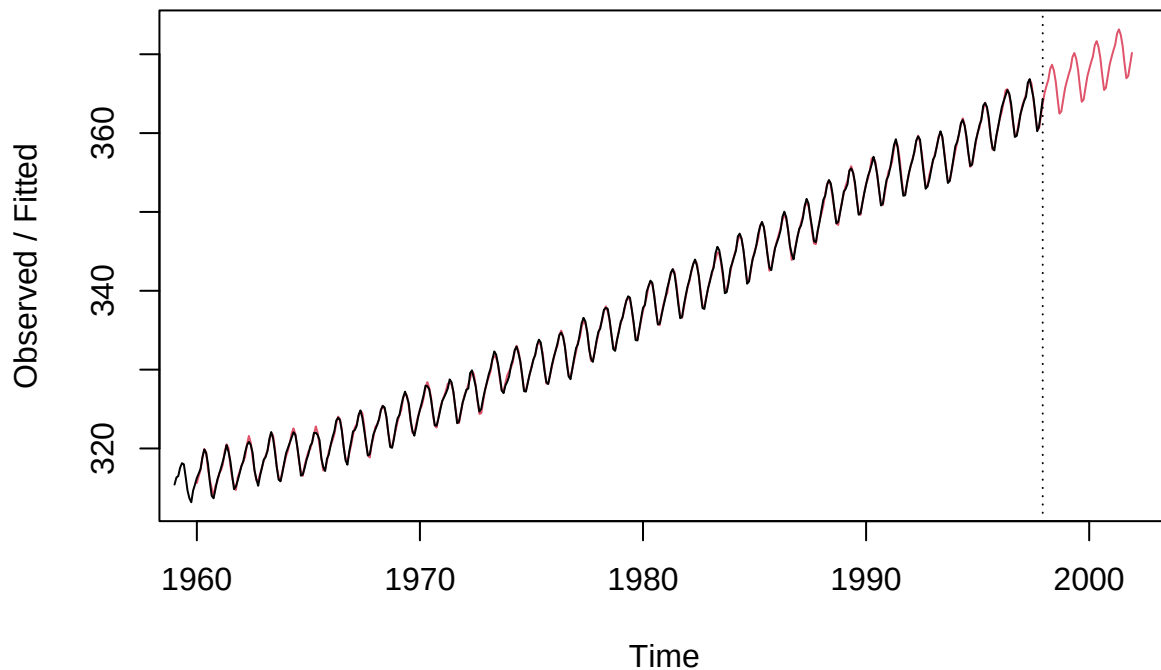
```
##      gamma
## 0.4728868
```

```
plot(y)
```



```
z<-predict(y,n.ahead=4*12)  
plot(y,z,lty=1:2, main = "Holt-Winters filtering and prediction")
```

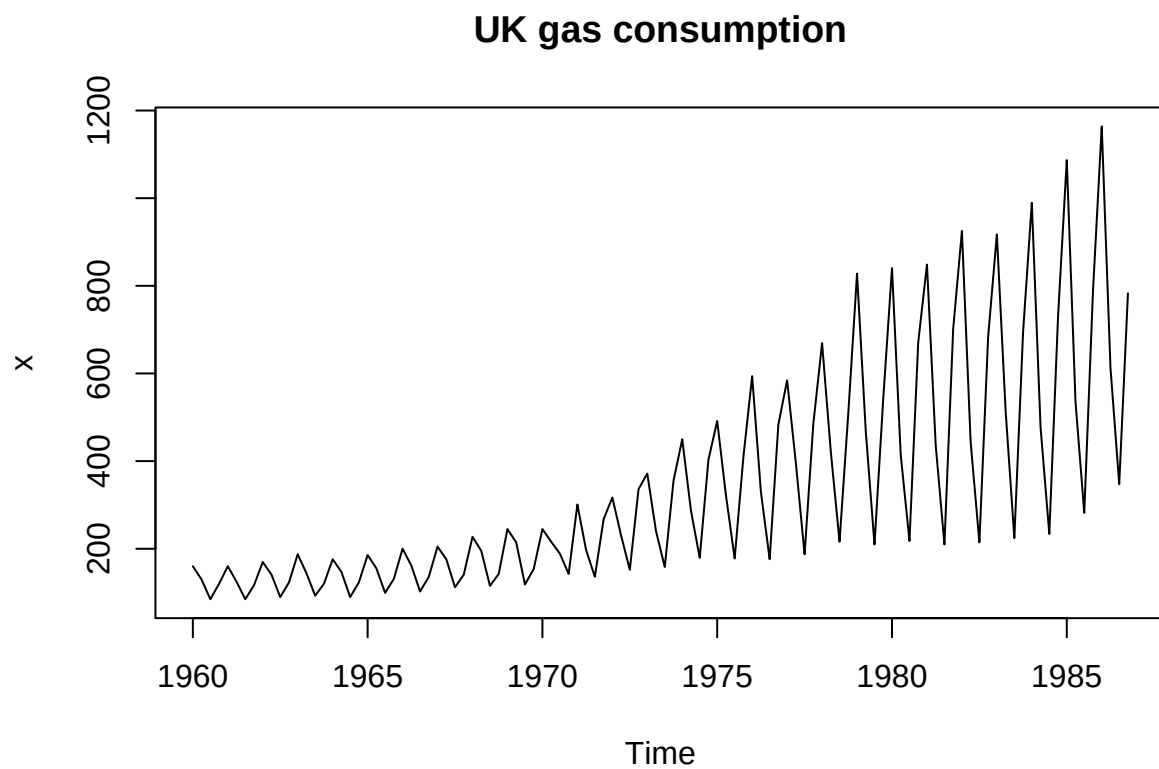
Holt-Winters filtering and prediction



The parameters “a”, “b”, “c” automatically selected create a fitting curve that describes well the original series (the red line maintains the level, trend and seasonality). As a consequence, the forecast produced for the next four years continues the same structural pattern, showing a consistent evolution of the level together with the repeating seasonal oscillations.

- UK gas consumption

```
# UK gas consumption  
x <- UKgas  
y <- HoltWinters(x)  
plot(x, main = "UK gas consumption")
```



```
y$alpha
```

```
##      alpha  
## 0.02127689
```

```
y$beta
```

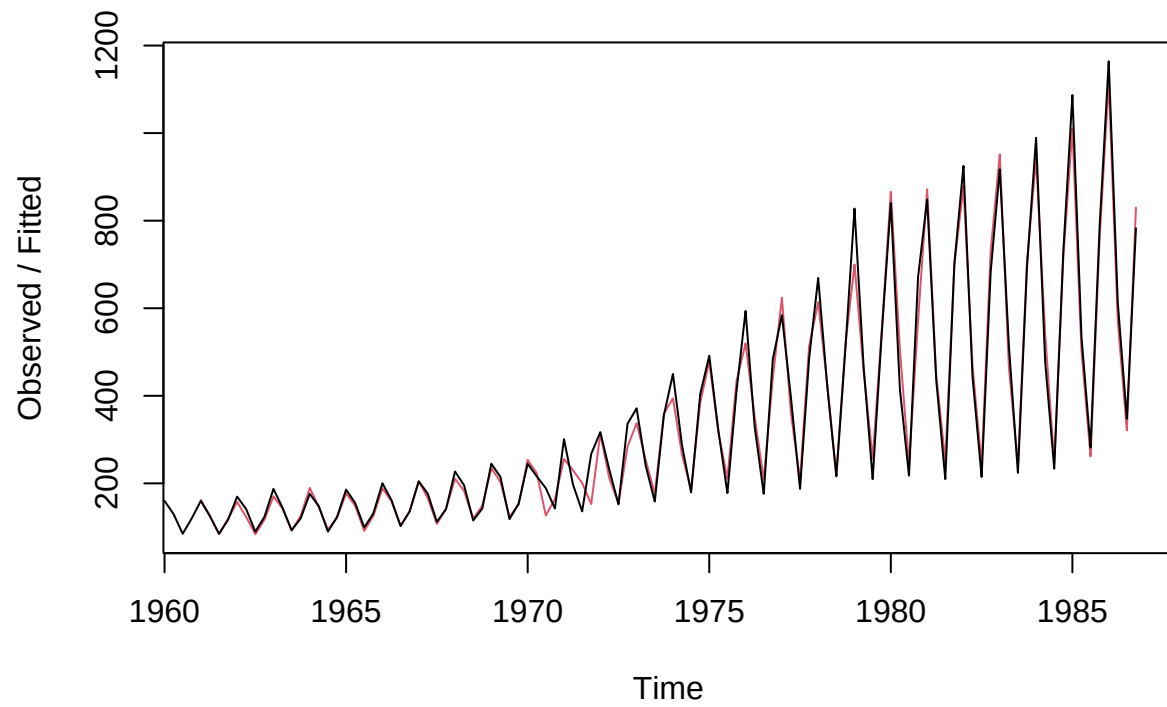
```
## beta  
##      1
```

```
y$gamma
```

```
##      gamma  
## 0.9899092
```

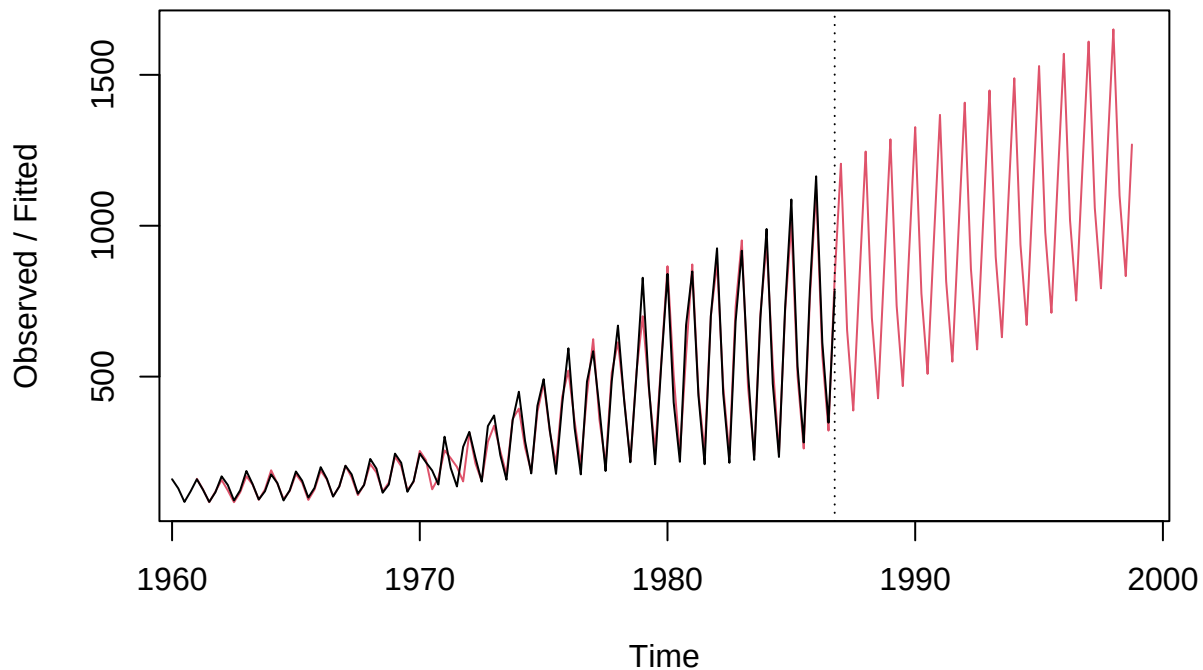
```
plot(y)
```

Holt-Winters filtering



```
z<-predict(y,n.ahead=4*12)
plot(y,z,lty=1:2, main = "Holt-Winters filtering and prediction")
```

Holt-Winters filtering and prediction



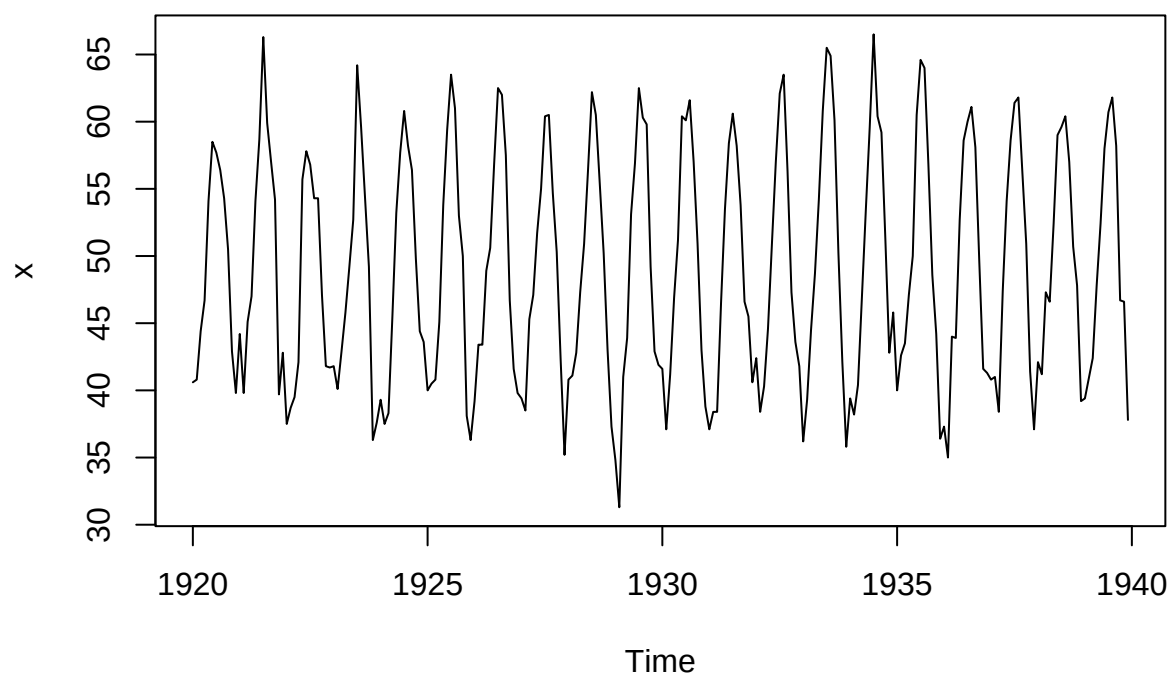
The series of *UK gas consumption* shows a strong increasing trend together with a seasonal pattern whose amplitude grows over time. The forecast continues the same structural behaviour: the predicted values show a continuation of the increasing trend together with seasonal oscillations whose amplitude keeps growing over time.

- Monthly temperature - Nottingham

```
# Monthly temperature - Nottingham
x <- nottem
y <- HoltWinters(x)

plot(x, main = "Monthly temperature - Nottingham")
```

Monthly temperature – Nottingham



```
y$alpha
```

```
##      alpha  
## 0.1383228
```

```
y$beta
```

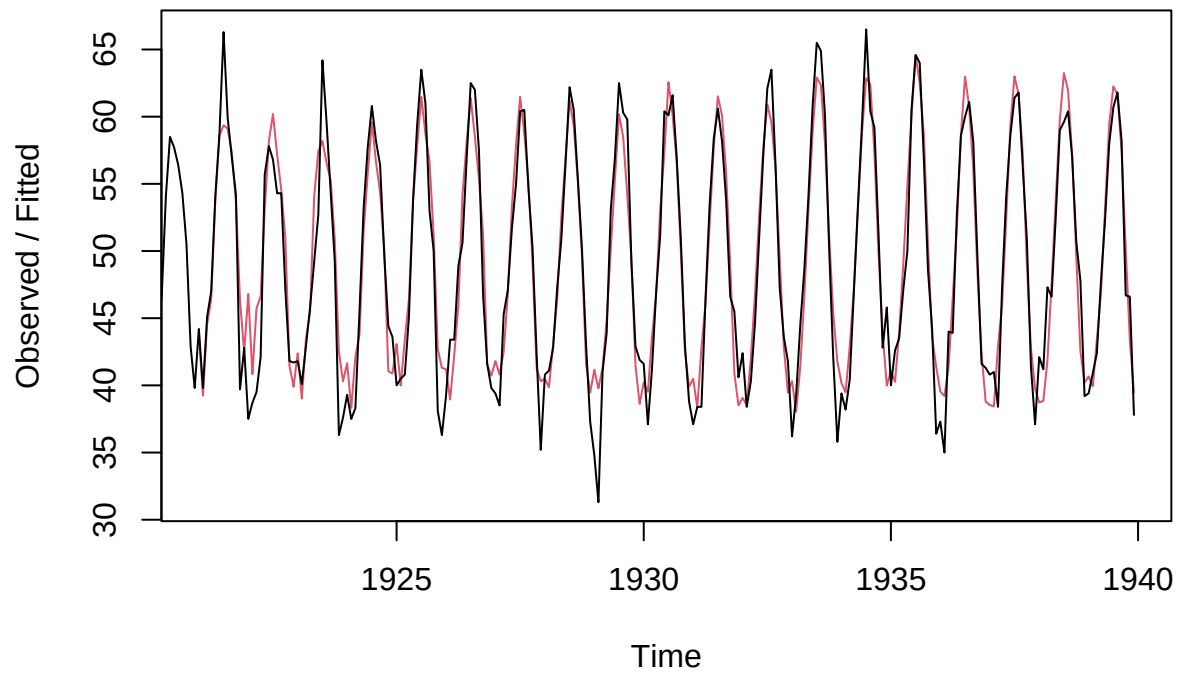
```
##      beta  
## 0.02327472
```

```
y$gamma
```

```
##      gamma  
## 0.2348744
```

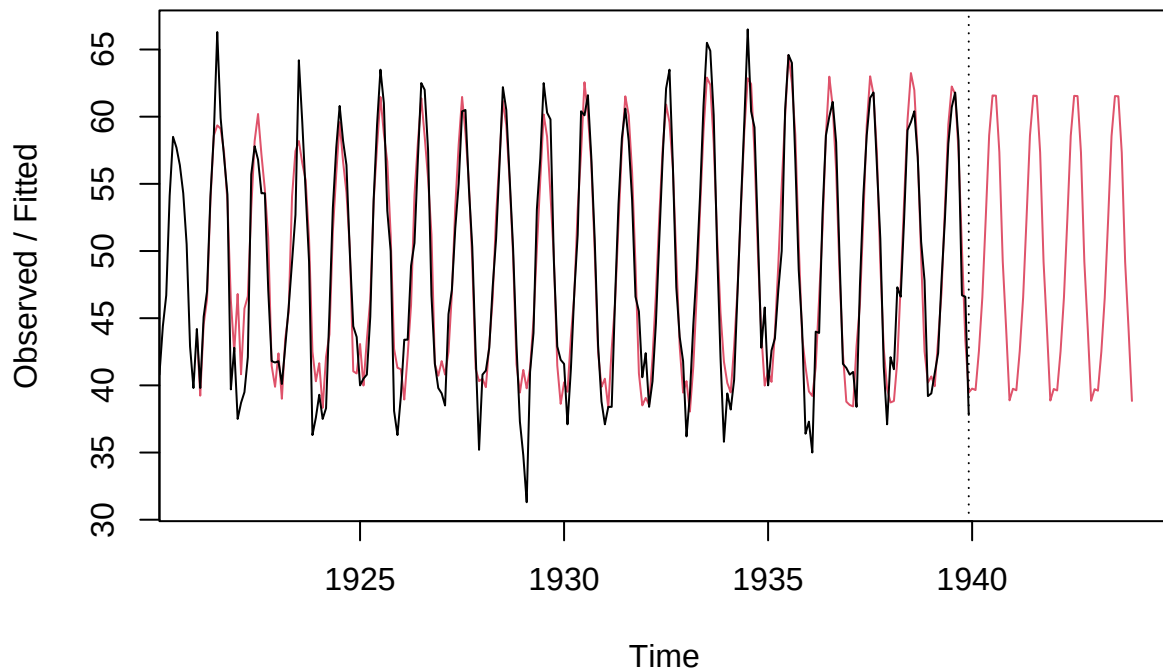
```
plot(y)
```


Holt-Winters filtering



```
z<-predict(y,n.ahead=4*12)
plot(y,z,lty=1:2, main = "Holt-Winters filtering and prediction")
```

Holt–Winters filtering and prediction



The *Monthly temperature - Nottingham* series shows a clear seasonal pattern but not a long-term trend (the fluctuations are around a relatively stable level). The Holt–Winters filtering captures this behavior well, and in fact the fitted curve follows the seasonal pattern while smoothing part of the irregular fluctuations in the original observations.